



(12) **United States Patent
Blankenship**

(10) **Patent No.: US 12,399,620 B2**
(45) **Date of Patent: Aug. 26, 2025**

(54) **IMPLIED DIRECTORY STATE UPDATES**

(71) Applicant: **Intel Corporation**, Santa Clara, CA
(US)

(72) Inventor: **Robert G. Blankenship**, Tacoma, WA
(US)

(73) Assignee: **Intel Corporation**, Santa Clara, CA
(US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 446 days.

(21) Appl. No.: **18/070,284**

(22) Filed: **Nov. 28, 2022**

(65) **Prior Publication Data**

US 2023/0367481 A1 Nov. 16, 2023

Related U.S. Application Data

(63) Continuation of application No. 16/372,247, filed on
Apr. 1, 2019, now abandoned, which is a continuation
(Continued)

(51) **Int. Cl.**
G06F 3/06 (2006.01)
G06F 12/0802 (2016.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06F 3/061** (2013.01); **G06F 3/0604**
(2013.01); **G06F 3/065** (2013.01); **G06F**
3/0671 (2013.01);
(Continued)

(58) **Field of Classification Search**
CPC G06F 3/061; G06F 3/0604; G06F 3/065;
G06F 3/0671; G06F 12/0802;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,606,686 A 2/1997 Tarui et al.
6,611,906 B1 8/2003 McAllister et al.
(Continued)

FOREIGN PATENT DOCUMENTS

CN 1252142 A 8/2005
CN 101859281 A 10/2010
(Continued)

OTHER PUBLICATIONS

EPO Summons to Attend Oral Proceedings pursuant to Rule 115(1)
EPC in EP Application Serial No. 16773666.9-1244 mailed on Dec.
1, 2022 (5 pages).

(Continued)

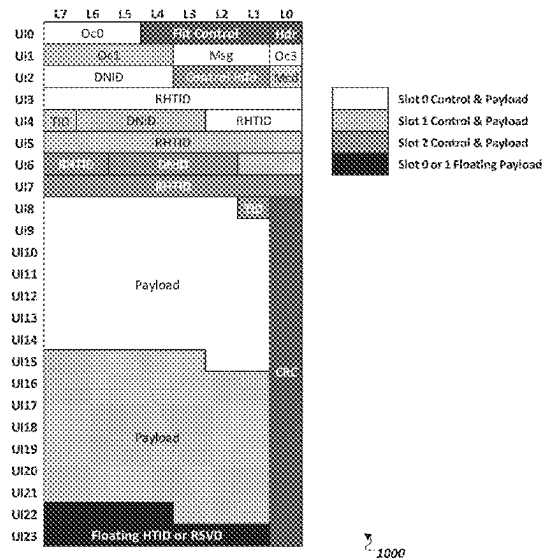
Primary Examiner — Tracy C Chan

(74) *Attorney, Agent, or Firm* — Nicholson De Vos
Webster & Elliott LLP

(57) **ABSTRACT**

A request is received over a link that requests a particular
line in memory. A directory state record is identified in
memory that identifies a directory state of the particular line.
A type of the request is identified from the request. It is
determined that the directory state of the particular line is to
change from the particular state to a new state based on the
directory state of the particular line and the type of the
request. The directory state record is changed, in response to
receipt of the request, to reflect the new state. A copy of the
particular line is sent in response to the request.

20 Claims, 22 Drawing Sheets



Related U.S. Application Data

of application No. 15/393,744, filed on Dec. 29, 2016, now Pat. No. 10,248,325, which is a continuation of application No. 14/670,526, filed on Mar. 27, 2015, now Pat. No. 9,639,276.

(51) Int. Cl.

G06F 12/0808 (2016.01)
G06F 12/0817 (2016.01)
G06F 12/128 (2016.01)
G06F 13/16 (2006.01)
G06F 12/0831 (2016.01)
G06F 12/12 (2016.01)

(52) U.S. Cl.

CPC **G06F 12/0802** (2013.01); **G06F 12/0808** (2013.01); **G06F 12/0824** (2013.01); **G06F 12/128** (2013.01); **G06F 13/1673** (2013.01); **G06F 12/0831** (2013.01); **G06F 12/12** (2013.01); **G06F 2212/1024** (2013.01); **G06F 2212/60** (2013.01); **G06F 2212/622** (2013.01); **Y02D 10/00** (2018.01)

(58) Field of Classification Search

CPC .. G06F 12/0808; G06F 12/0824; G06F 12/12; G06F 12/128; G06F 12/0831; G06F 13/0673; G06F 2212/1024; G06F 2212/60; G06F 2212/622

See application file for complete search history.

(56)**References Cited****U.S. PATENT DOCUMENTS**

6,772,288	B1	8/2004	Flake et al.
7,114,043	B2	9/2006	Rowlands
7,343,456	B2	3/2008	Rowlands
9,432,298	B1	8/2016	Smith
9,817,693	B2	11/2017	Busaba et al.
2002/0010840	A1	1/2002	Barroso et al.
2004/0030754	A1	2/2004	Ree
2004/0148472	A1	7/2004	Barroso et al.
2005/0223126	A1	10/2005	Chu et al.
2006/0179241	A1	8/2006	Clark et al.
2008/0086602	A1	4/2008	Guthrie et al.
2008/0104293	A1	5/2008	Briggs et al.
2010/0293420	A1	11/2010	Kapil et al.
2013/0262778	A1	10/2013	Ghai et al.
2013/0332696	A1	12/2013	Schmidt et al.
2014/0040550	A1	2/2014	Nale et al.
2014/0112339	A1*	4/2014	Safraneck G06F 13/4286 370/389
2014/0114887	A1	4/2014	Iyer et al.
2014/0115207	A1	4/2014	Iyer et al.

2014/0115374	A1	4/2014	Iyer et al.
2014/0310471	A1	10/2014	Hollaway, Jr. et al.
2015/0006762	A1	1/2015	Lee et al.
2015/0033051	A1	1/2015	Gendler et al.
2015/0378895	A1	12/2015	Gschwind et al.
2016/0179667	A1	6/2016	Kumar et al.
2016/0373557	A1*	12/2016	Sikkink H04L 69/22
2017/0109300	A1*	4/2017	Halleck G06F 13/1678
2018/0241506	A1*	8/2018	Kohda H04L 1/0061

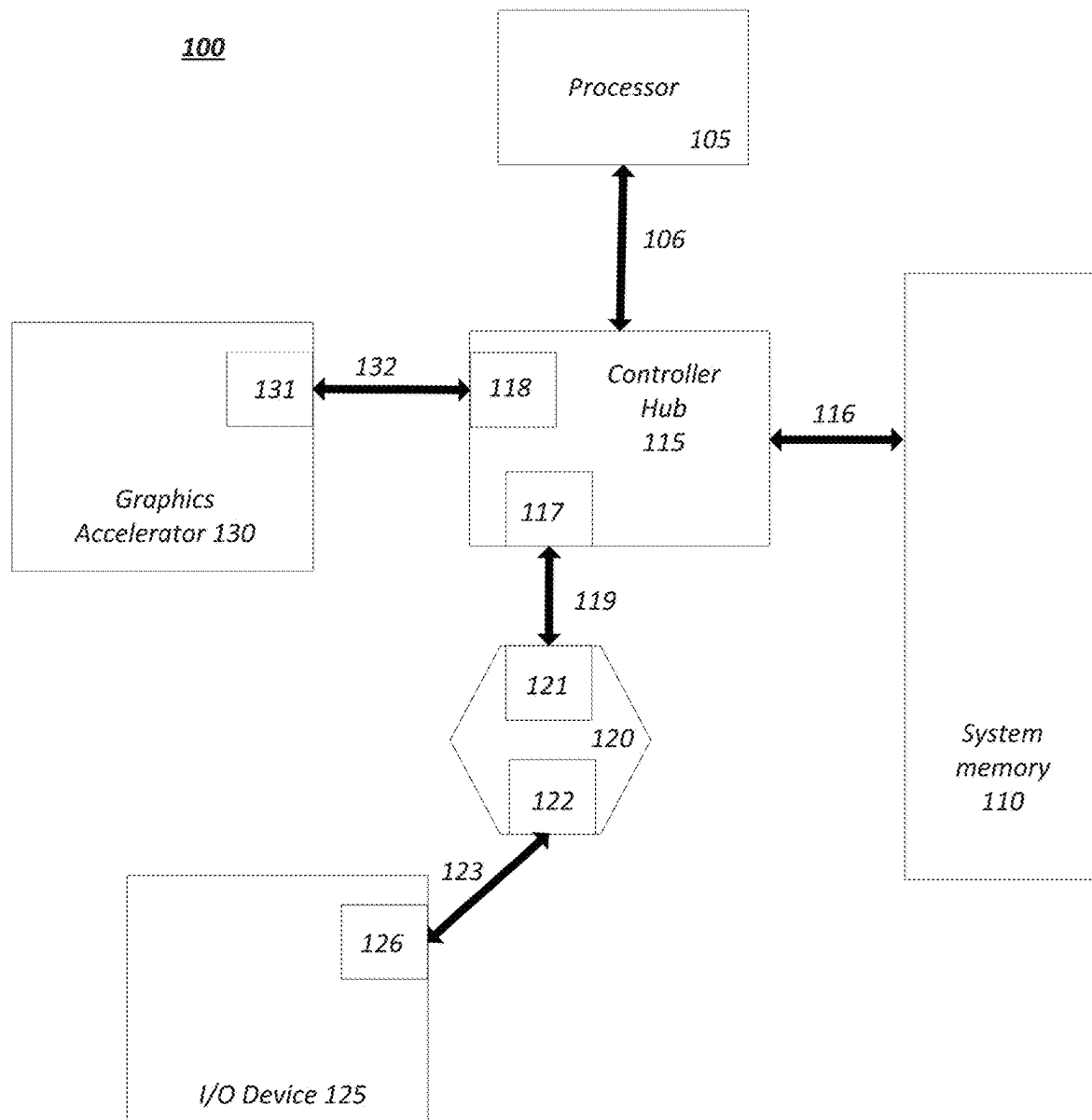
FOREIGN PATENT DOCUMENTS

CN	101093464	A	8/2011
CN	103119568	A	5/2013
CN	103154909	A	6/2013
CN	103870395	A	6/2014
CN	104380269	A	1/2018
TW	200919190	A	5/2009
WO	2013086060	A1	6/2013
WO	2016160199	A1	10/2016

OTHER PUBLICATIONS

EPO; Extended European Search Report issued in EP Patent Application No. 20197596.8, dated Mar. 10, 2021; 9 pages.
 EPO; Office Action issued in EP Patent Application No. 16773666.9, dated Apr. 14, 2021; 5 pages.
 Extended European Search Report for Application No. EP16773666.9, mailed on Nov. 8, 2018, 8 pages.
 International Preliminary Report on Patentability for Application No. PCT/US2016/019679, mailed on Oct. 12, 2017, 11 Pages.
 International Search Report and Written Opinion in International Application No. PCT/US2016/019679 mailed Jun. 8, 2016.
 Non Final Office Action in U.S. Appl. No. 14/670,526 dated Jun. 17, 2016 (18 pages).
 Notice of Allowance in U.S. Appl. No. 14/670,526 dated Dec. 27, 2016 (12 pages).
 SIPO; First Office Action issued in Chinese Patent Application No. CN 201680018780.X, dated Jun. 2, 2020; 30 pages with English translation.
 Taiwan Official Letter and Search Report in Taiwan Application No. 105103551 mailed Oct. 20, 2016.
 Taiwan Patent Office Notice of Allowance in Taiwan Application No. 105103551 dated Apr. 28, 2017, 3 pages.
 Taiwan Patent Office Search Report of TW Patent Application No. 106102714 dated Feb. 26, 2018, with English translation, 2 pages.
 China Patent Office First Office Action issued in Chinese Patent Application No. CN 202011523261.3, dated Dec. 25, 2023; 24 pages with English translation.
 China Patent Office Office Action issued in Chinese Patent Application No. CN 202011523261.3, dated Jun. 13, 2024; pages with English summary.
 China Patent Office Notice of Allowance for Chinese Patent Application No. CN202011523261.3 mailed on Aug. 30, 2024, 10 Pages—English Translation Available.

* cited by examiner

**FIG. 1**

Layered Protocol Stack 200

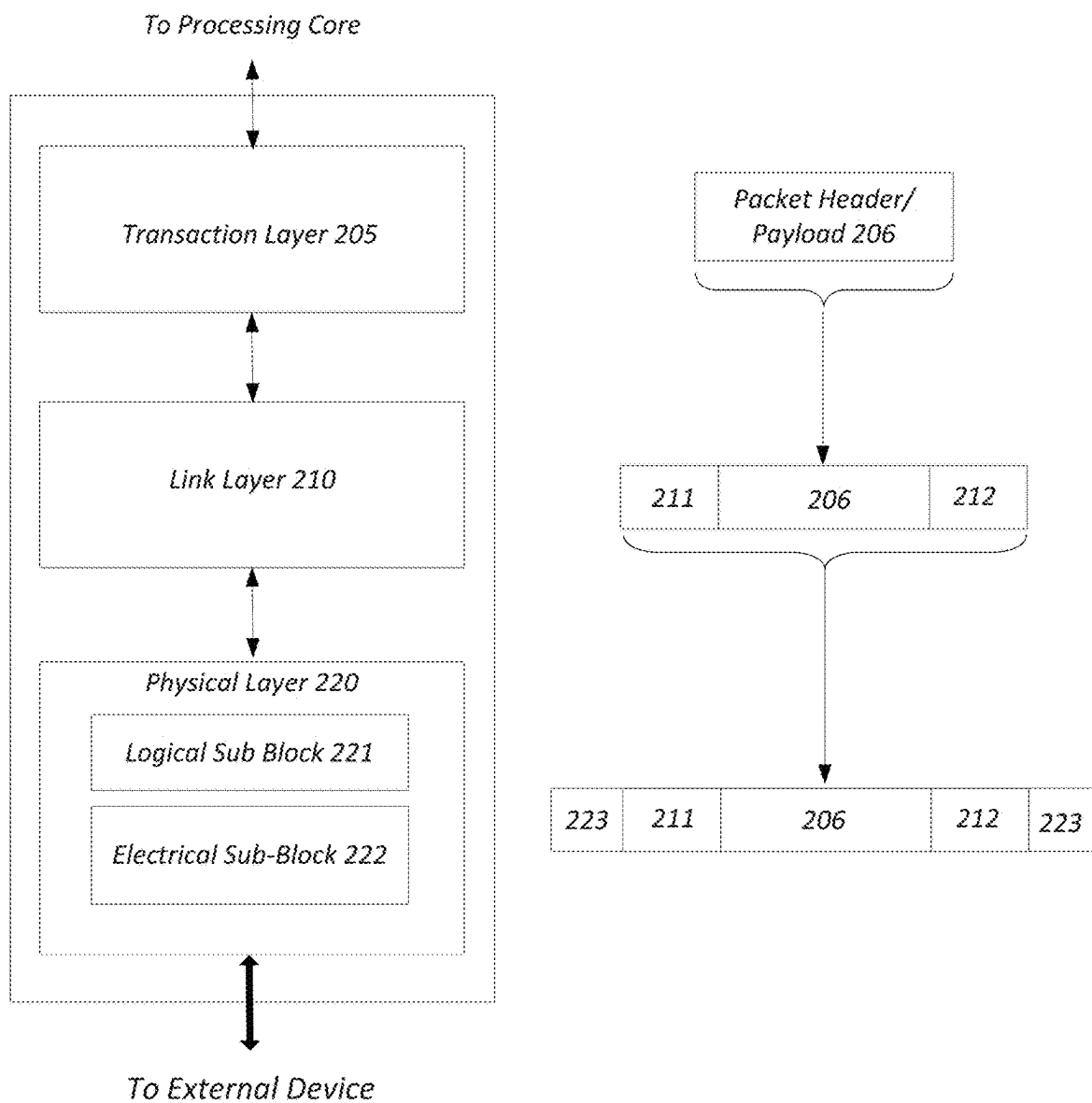


FIG. 2

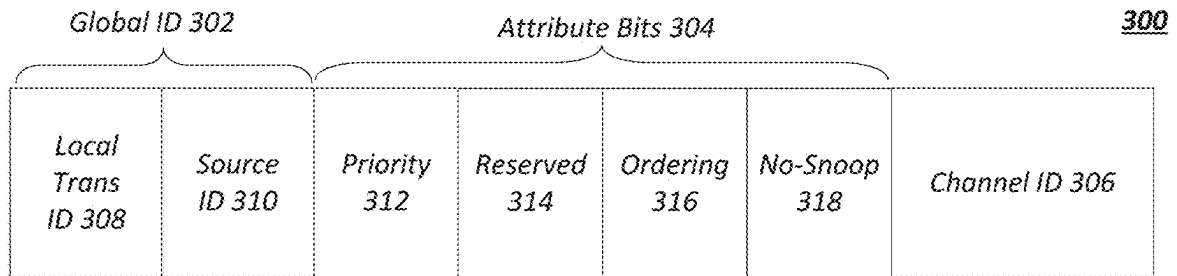


FIG. 3

400

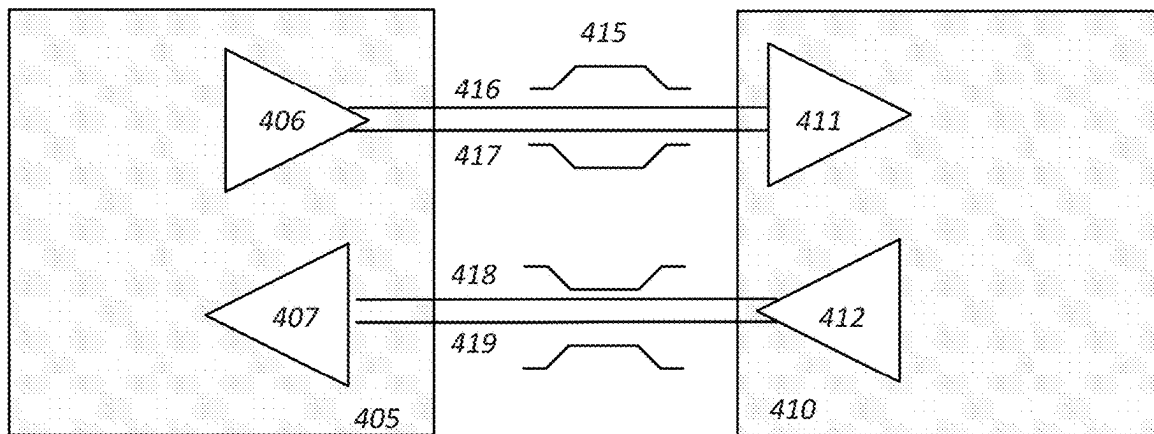


FIG. 4

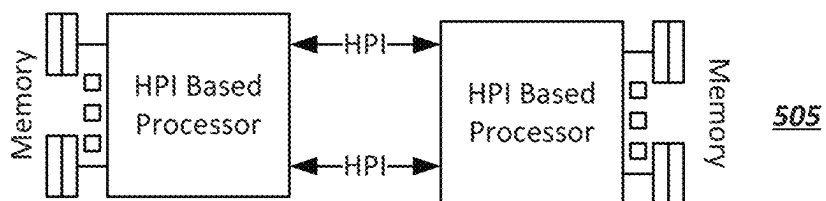
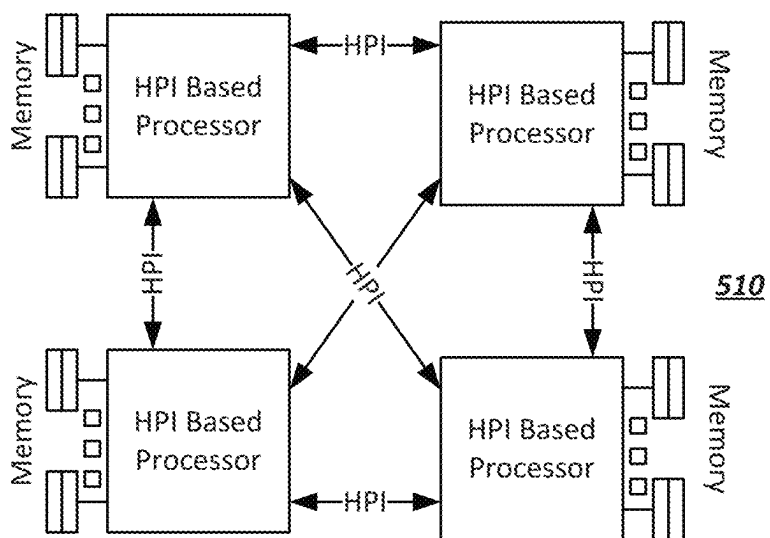
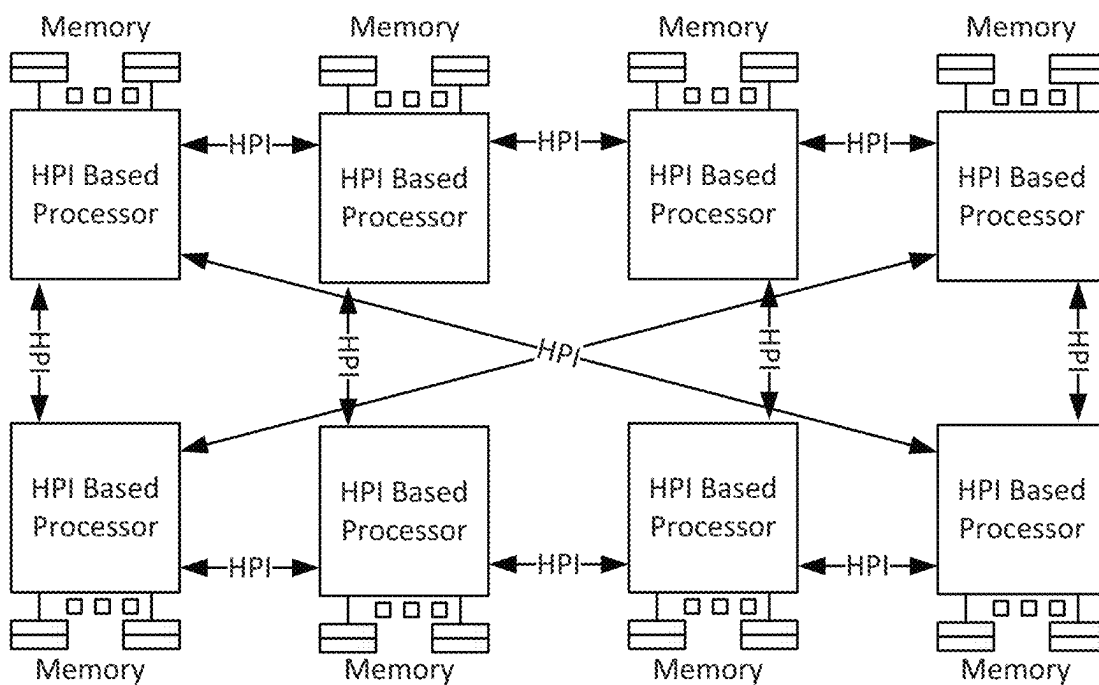


FIG. 5



515



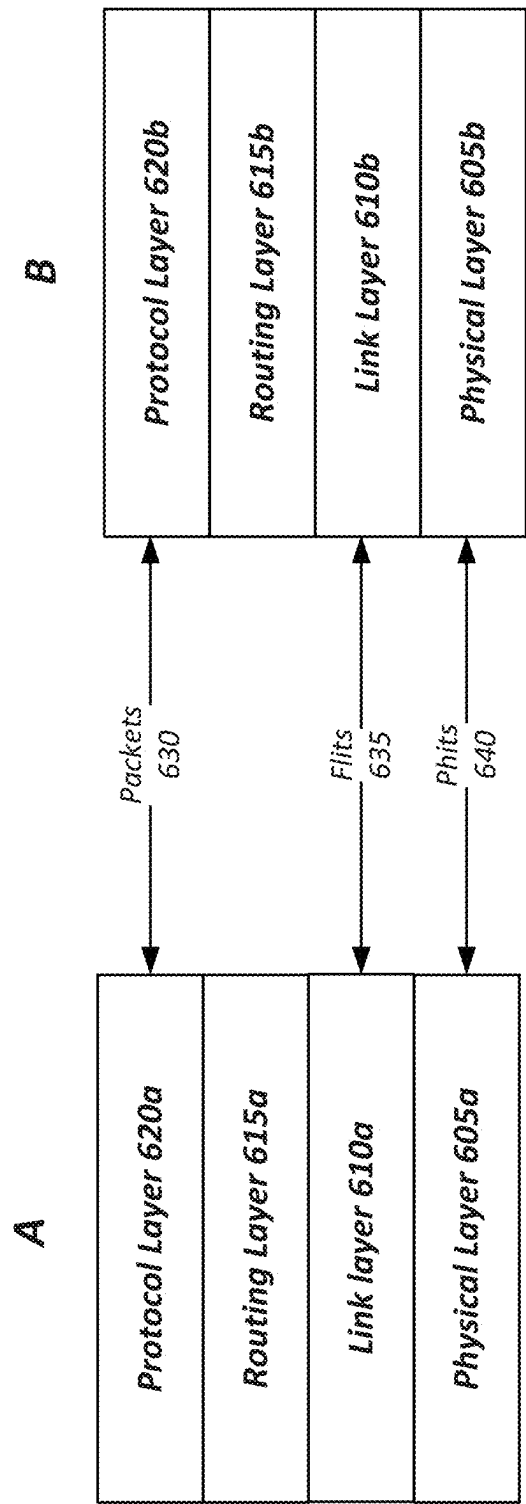
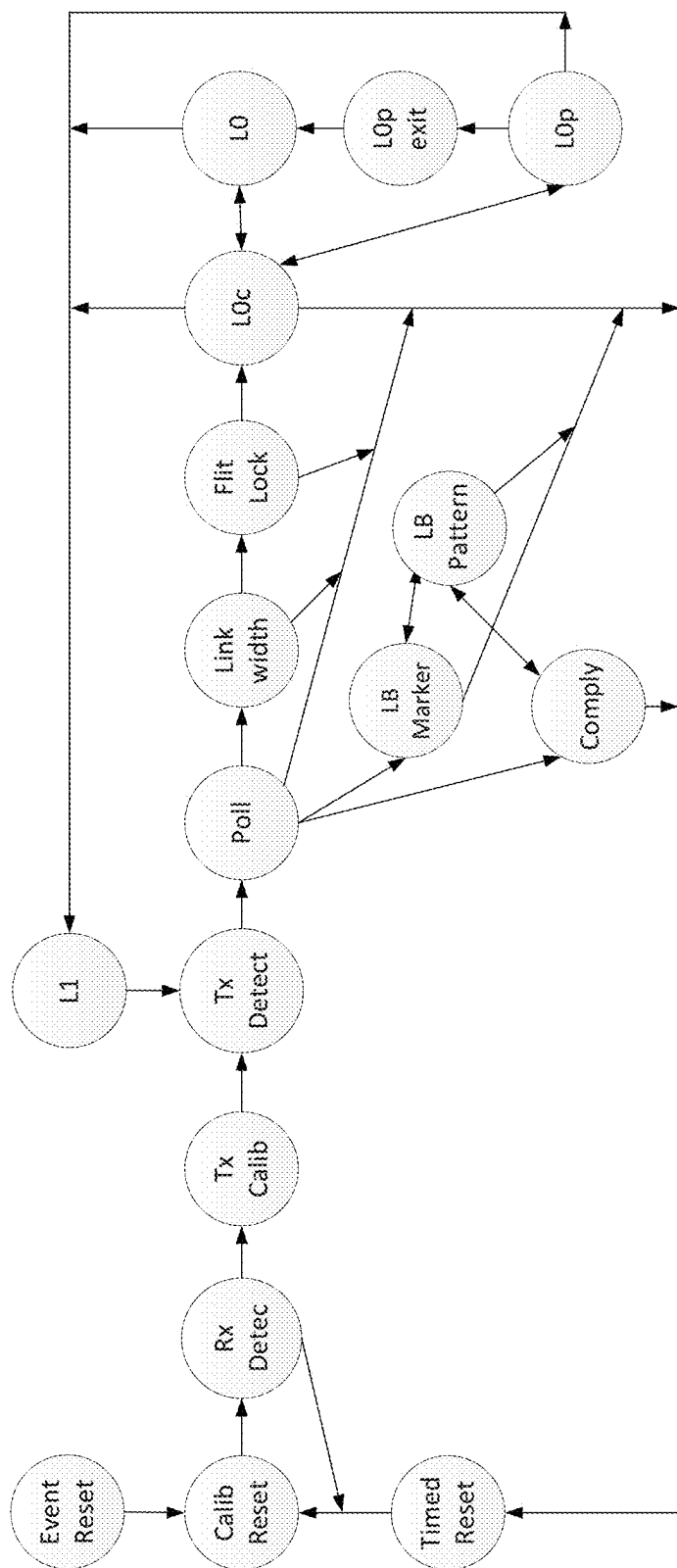


FIG. 6



700

FIG. 7

UI	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0, 1, 2, 3	47	44	42	39	37	34	32	29	27	24	22	19	17	14	12	9	7	5	3	1
4, 5, 6, 7	46	43	41	38	36	33	31	28	26	23	21	18	16	13	11	8	6	4	2	0
8, 9, 10, 11	45	45	40	40	35	35	30	30	25	25	20	20	15	16	10	12	9	7	4	2
12, 13, 14, 15	47	44	42	39	37	34	32	29	27	24	22	19	17	15	13	11	8	6	3	1
16, 17, 18, 19	46	43	41	38	36	33	31	28	26	23	21	18	19	14	14	10	9	5	4	0
20, 21, 22, 23	47	44	42	39	37	34	32	29	27	25	23	21	18	16	13	11	8	6	3	1
24, 25, 26, 27	46	43	41	38	36	33	31	28	26	24	22	20	17	15	12	10	7	5	2	0
28, 29, 30, 31	45	45	40	40	35	36	30	32	29	27	24	22	19	17	14	12	9	7	4	2
32, 33, 34, 35	47	44	42	39	37	35	33	31	28	26	23	21	18	16	13	11	8	6	3	1
36, 37, 38, 39	46	43	41	38	39	34	34	30	29	25	24	20	19	15	14	10	9	5	4	0
40, 41, 42, 43	47	45	43	41	38	36	33	31	28	26	23	21	18	16	13	11	8	6	3	1
44, 45, 46, 47	46	44	42	40	37	35	32	30	27	25	22	20	17	15	12	10	7	5	2	0

FIG. 8

800

UI								
	7	6	5	4	3	2	1	0
0, 1, 2, 3	47	41	35	29	23	17	11	5
4, 5, 6, 7	46	40	34	28	22	16	10	4
8, 9, 10, 11	45	39	33	27	21	15	9	3
12, 13, 14, 15	44	38	32	26	20	14	8	2
16, 17, 18, 19	43	37	31	25	19	13	7	1
20, 21, 22, 23	42	36	30	24	18	12	6	0

FIG. 9

Σ₉₀₀

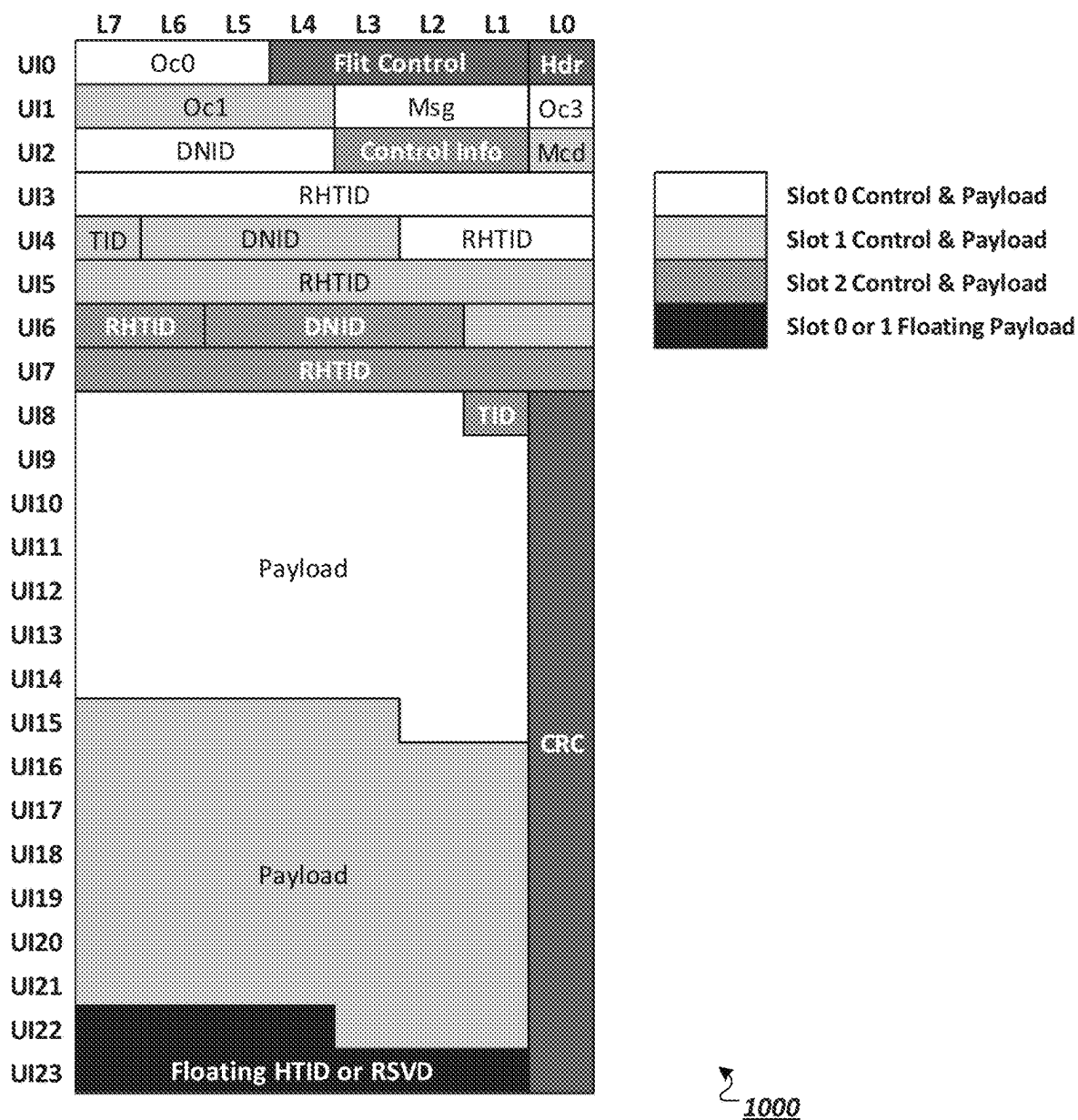


FIG. 10

1100 ↘

	L7	L6	L5	L4	L3	L2	L1	L0
UI0	191	167	143	119	95	71	47	23
UI1	190	166	142	118	94	70	46	22
UI2	189	165	141	117	93	69	45	21
UI3	188	164	140	116	92	68	44	20
UI4	187	163	139	115	91	67	43	19
UI5	186	162	138	114	90	66	42	18
UI6	185	161	137	113	89	65	41	17
UI7	184	160	136	112	88	64	40	16
UI8	183	159	135	111	87	63	39	15
UI9	182	158	134	110	86	62	38	14
UI10	181	157	133	109	85	61	37	13
UI11	180	156	132	108	84	60	36	12
UI12	179	155	131	107	83	59	35	11
UI13	178	154	130	106	82	58	34	10
UI14	177	153	129	105	81	57	33	9
UI15	176	152	128	104	80	56	32	8
UI16	175	151	127	103	79	55	31	7
UI17	174	150	126	102	78	54	30	6
UI18	173	149	125	101	77	53	29	5
UI19	172	148	124	100	76	52	28	4
UI20	171	147	123	99	75	51	27	3
UI21	170	146	122	98	74	50	26	2
UI22	169	145	121	97	73	49	25	1
UI23	168	144	120	96	72	48	24	0

FIG. 11

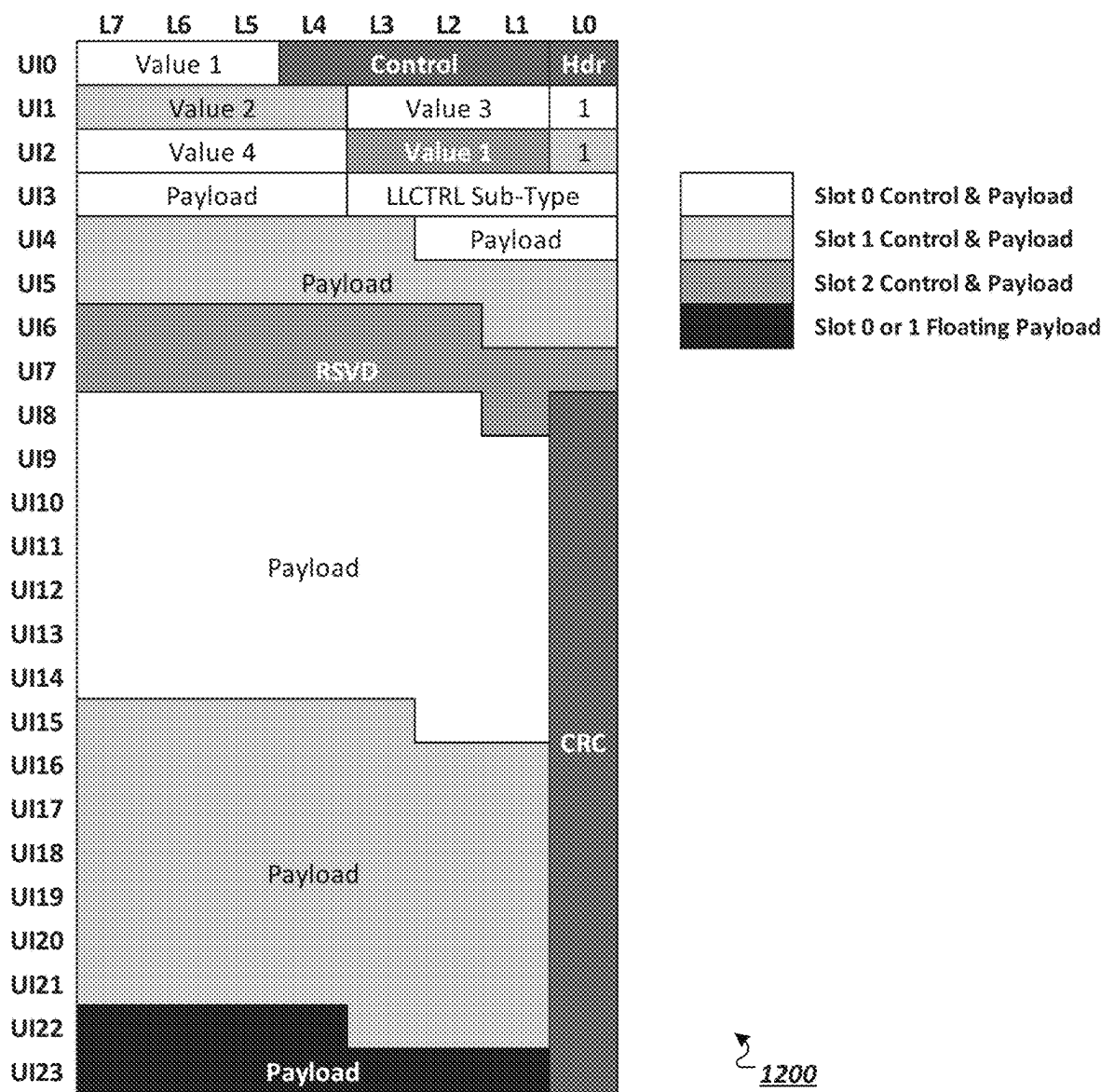


FIG. 12

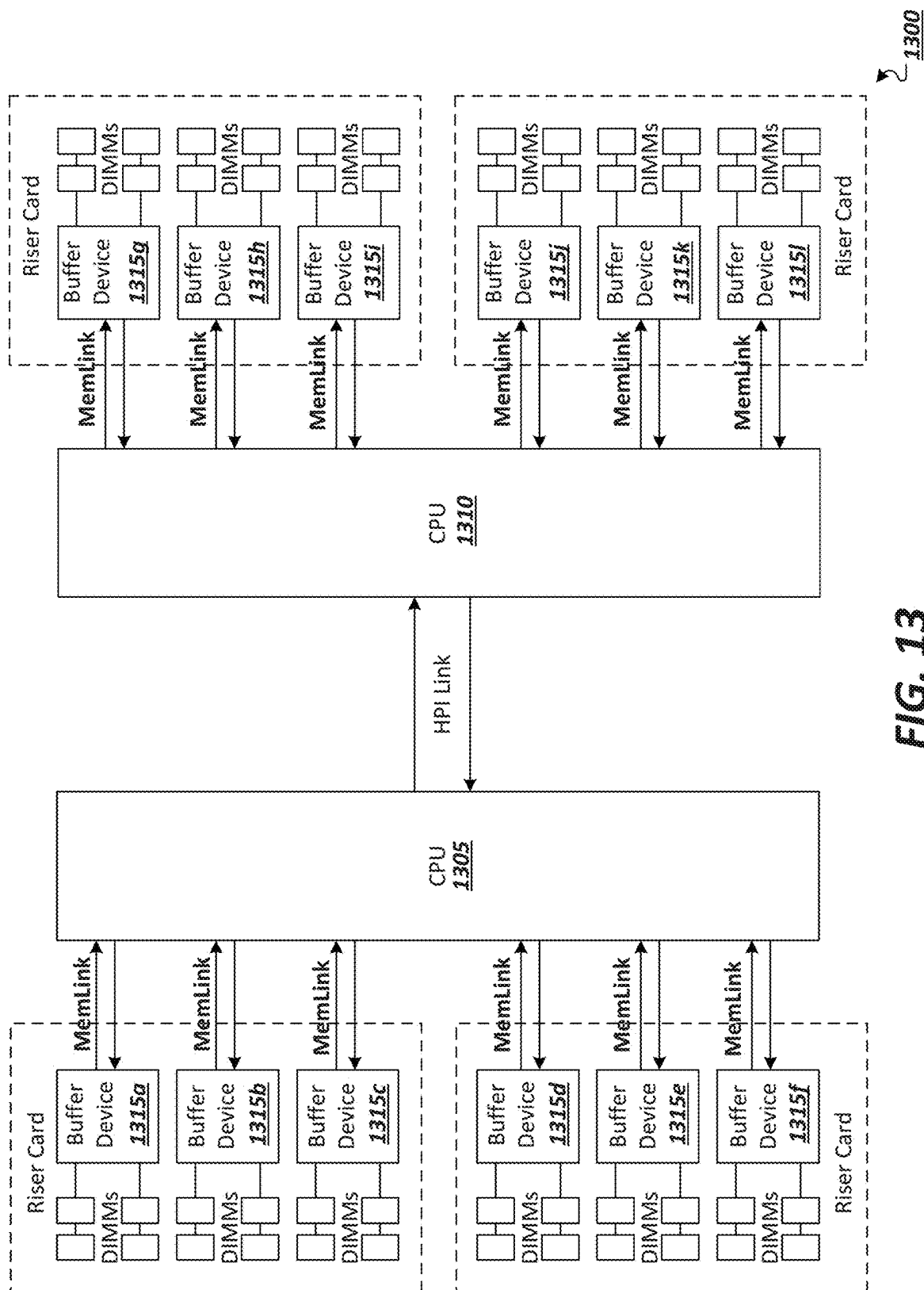


FIG. 13

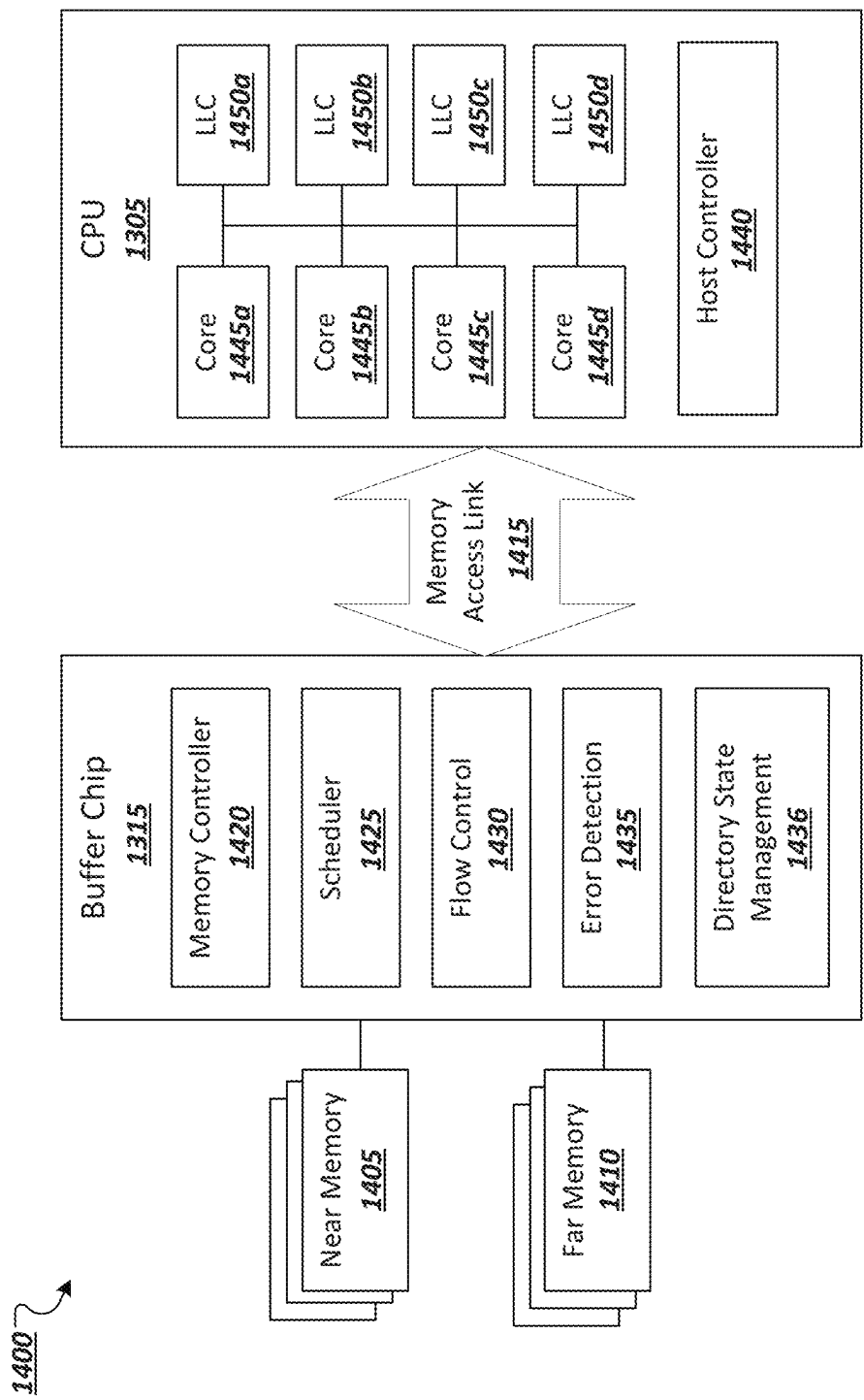


FIG. 14

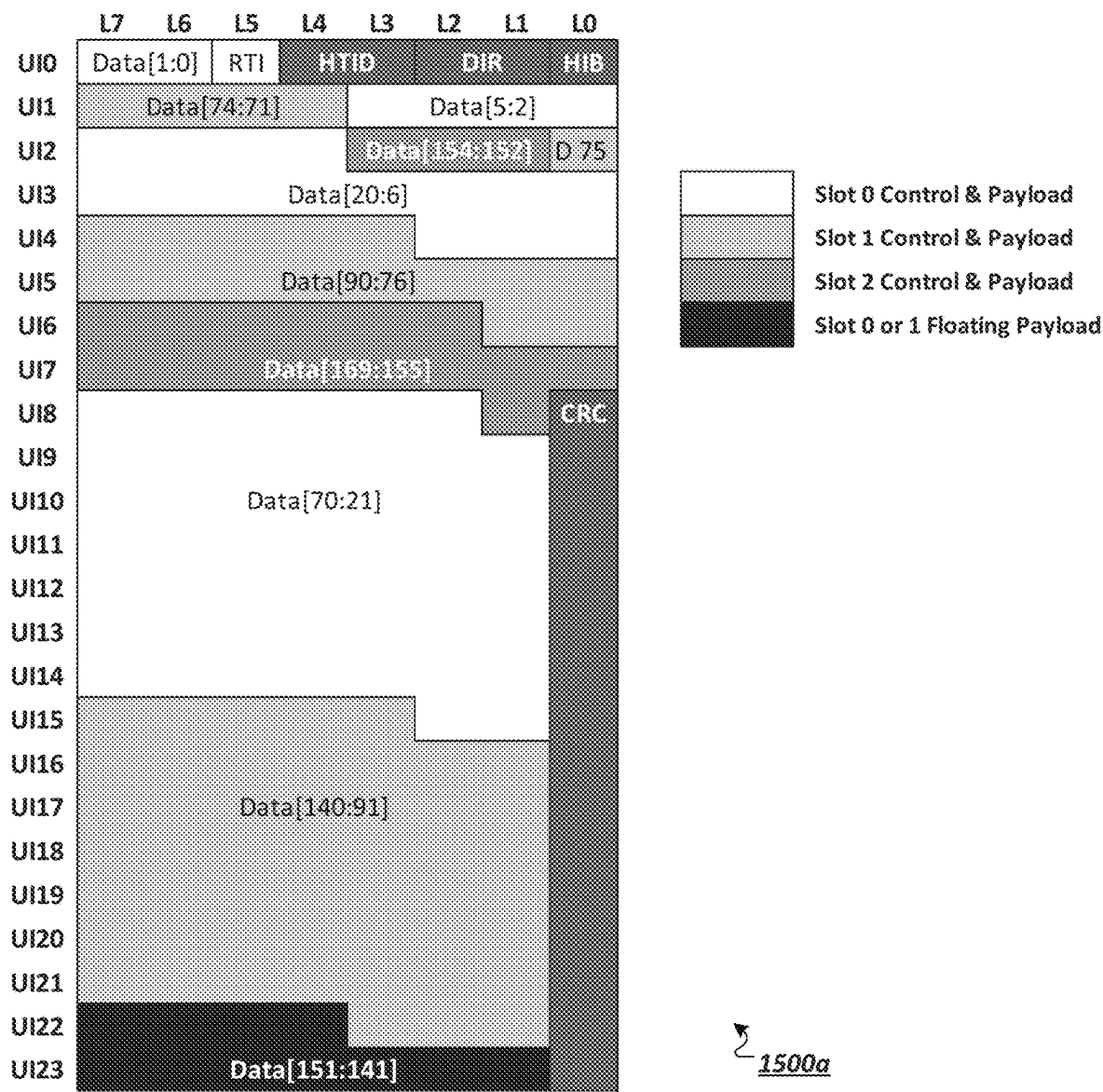


FIG. 15A

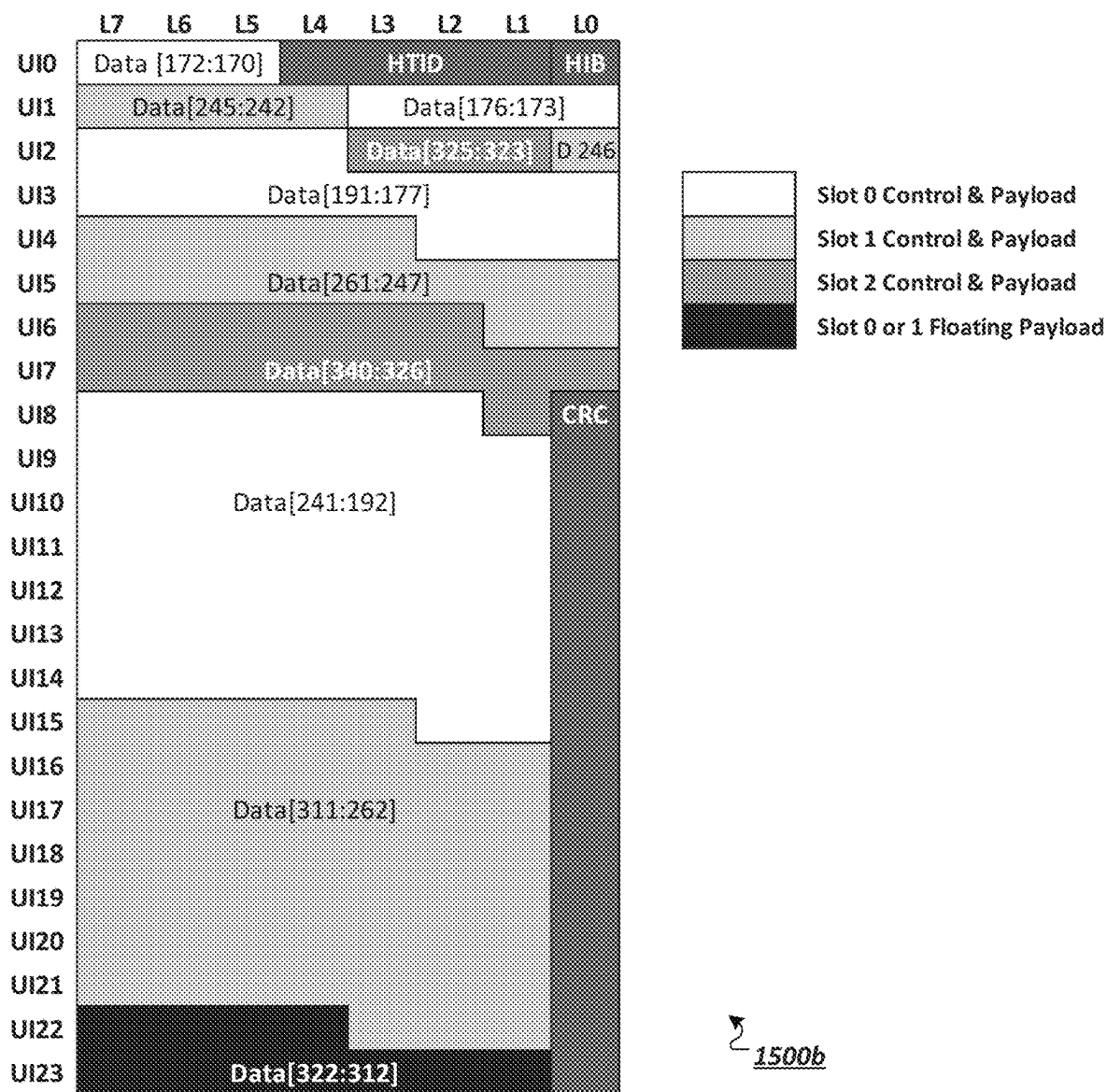


FIG. 15B

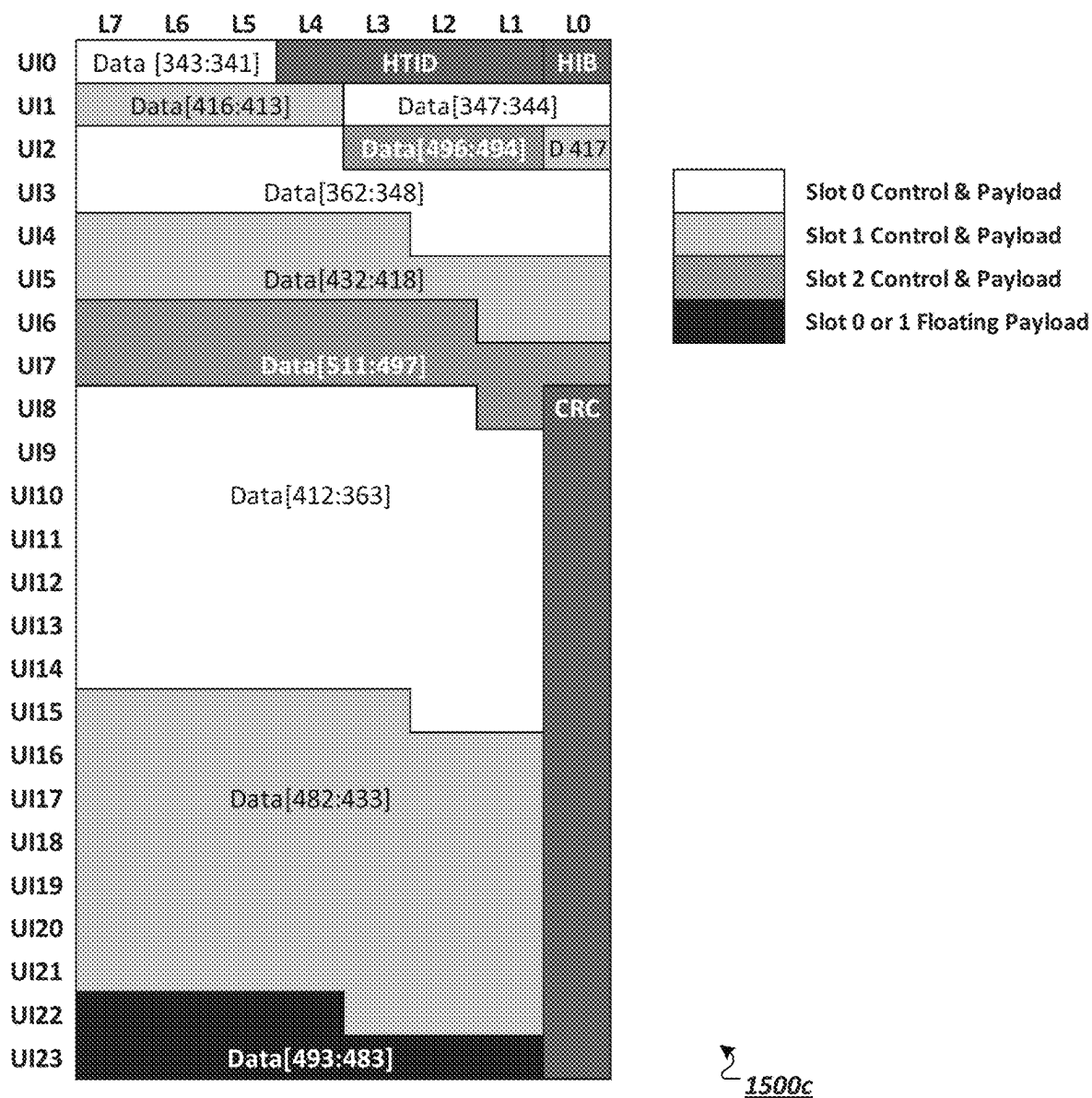


FIG. 15C

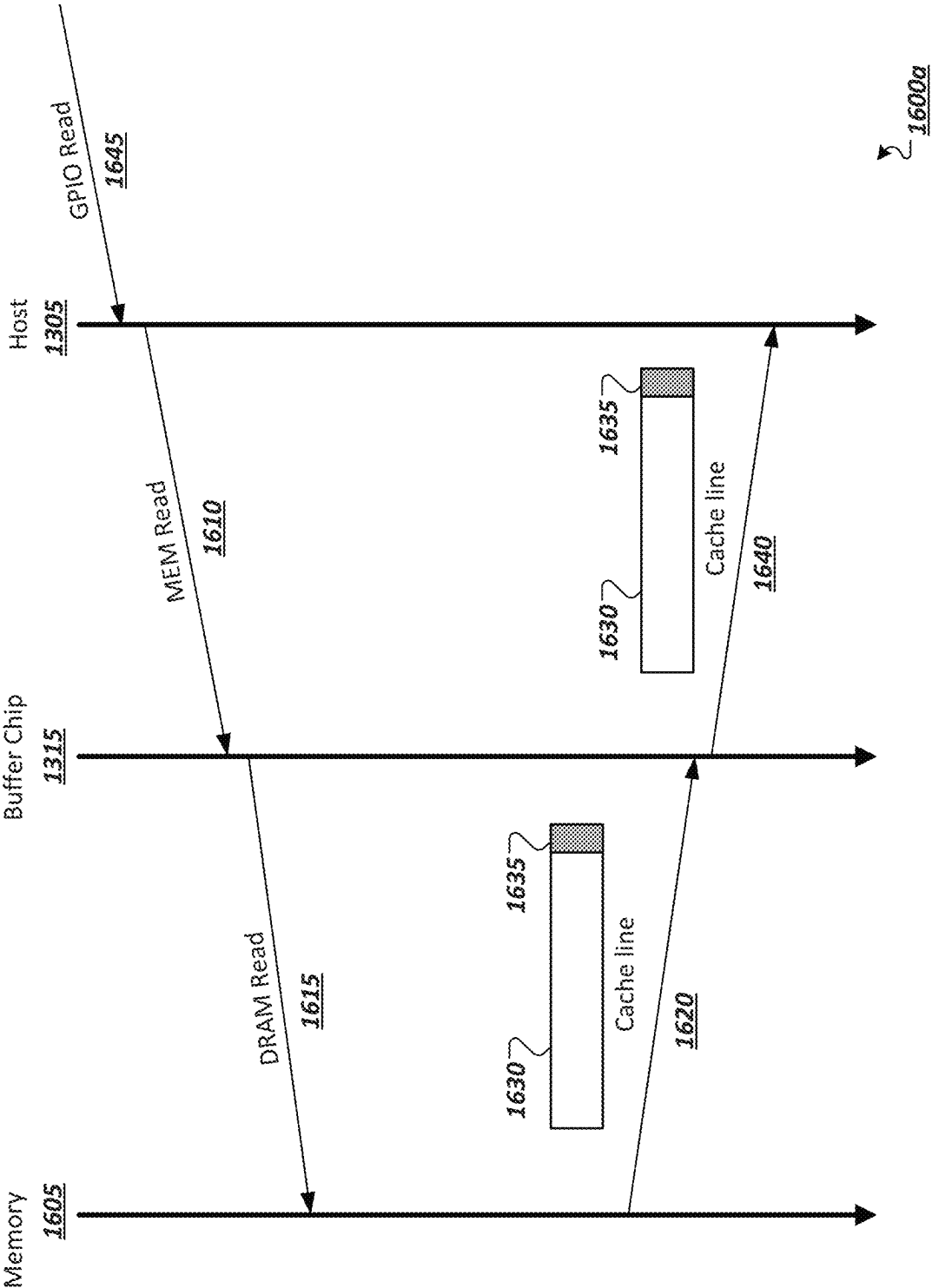


FIG. 16A

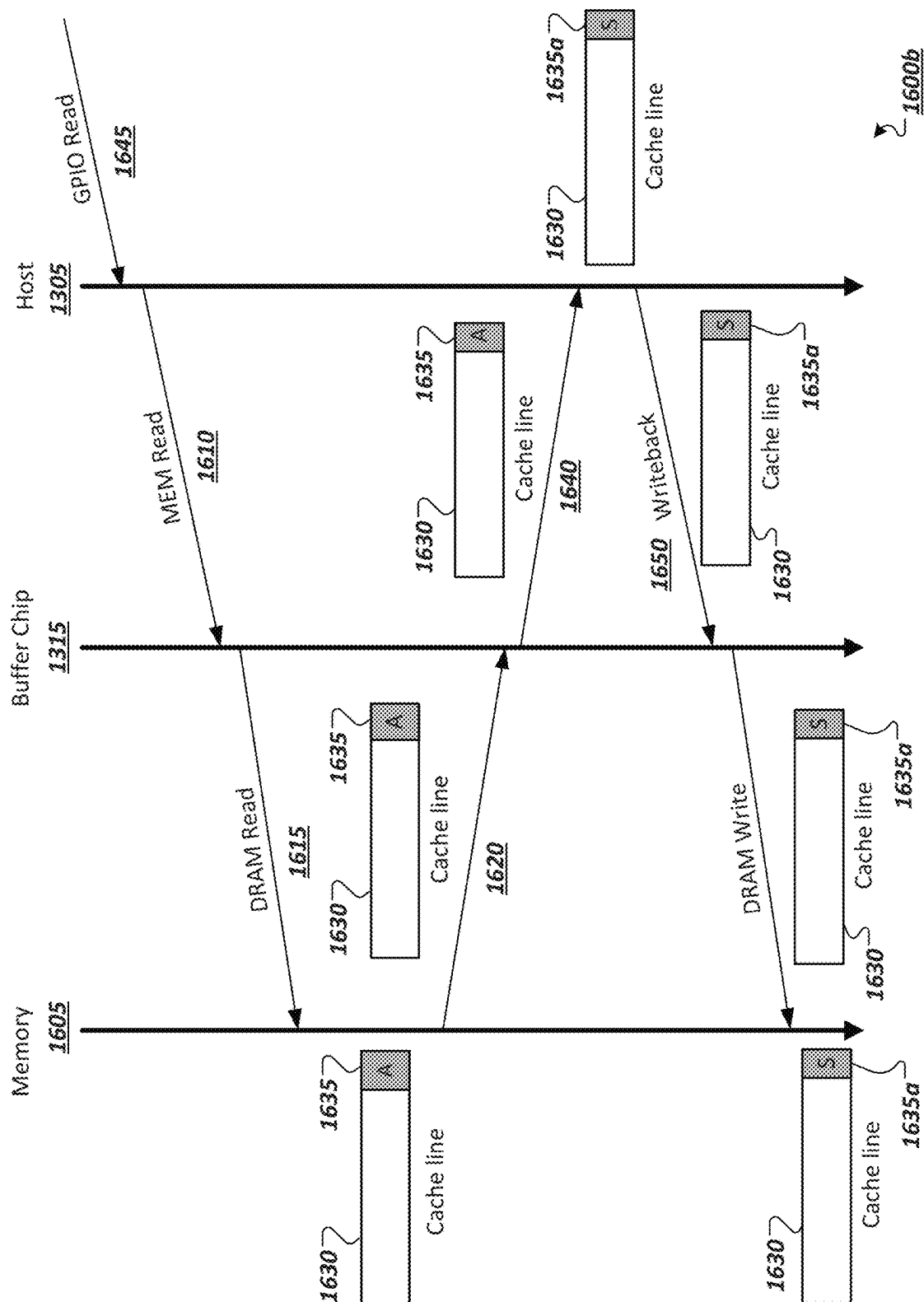
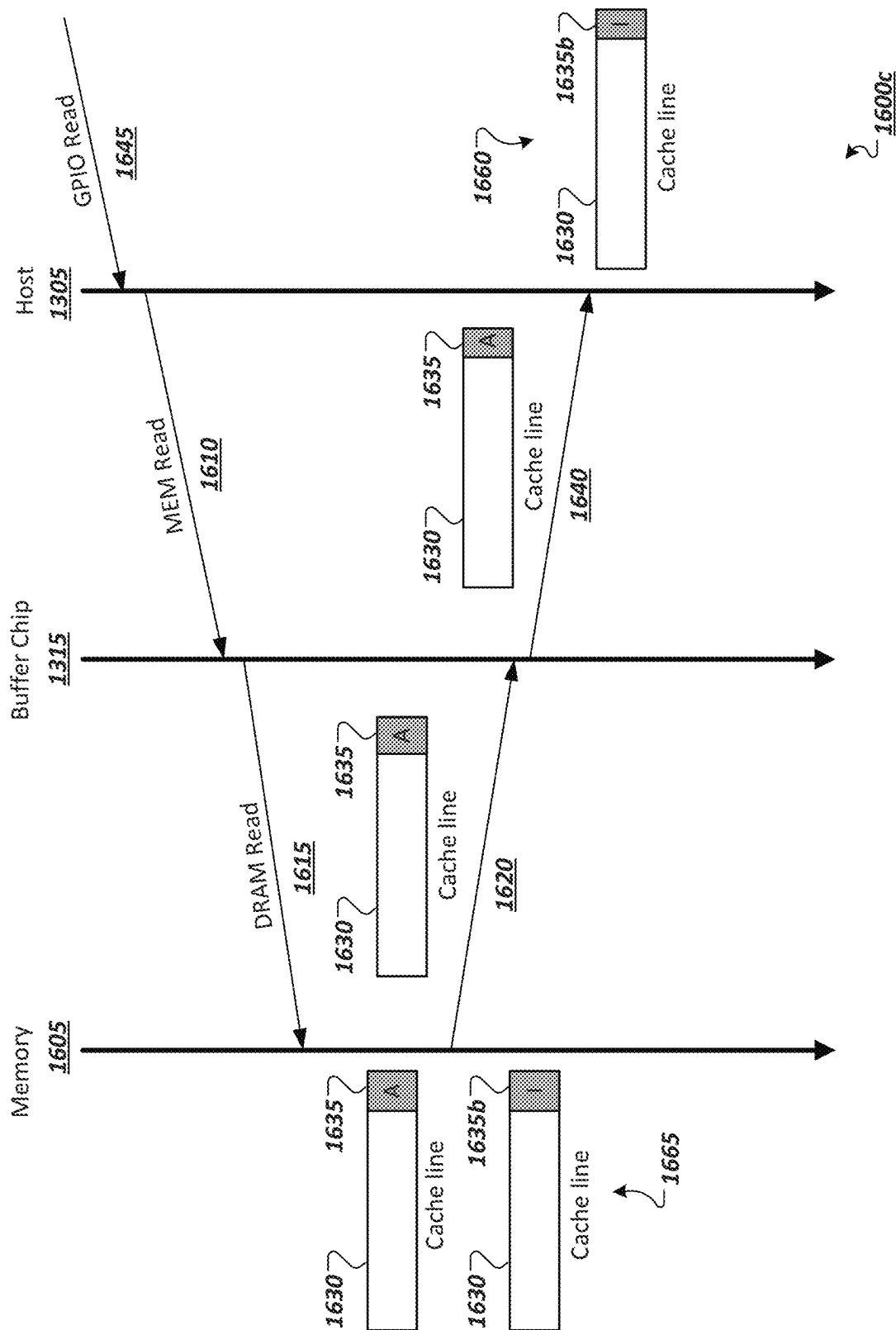
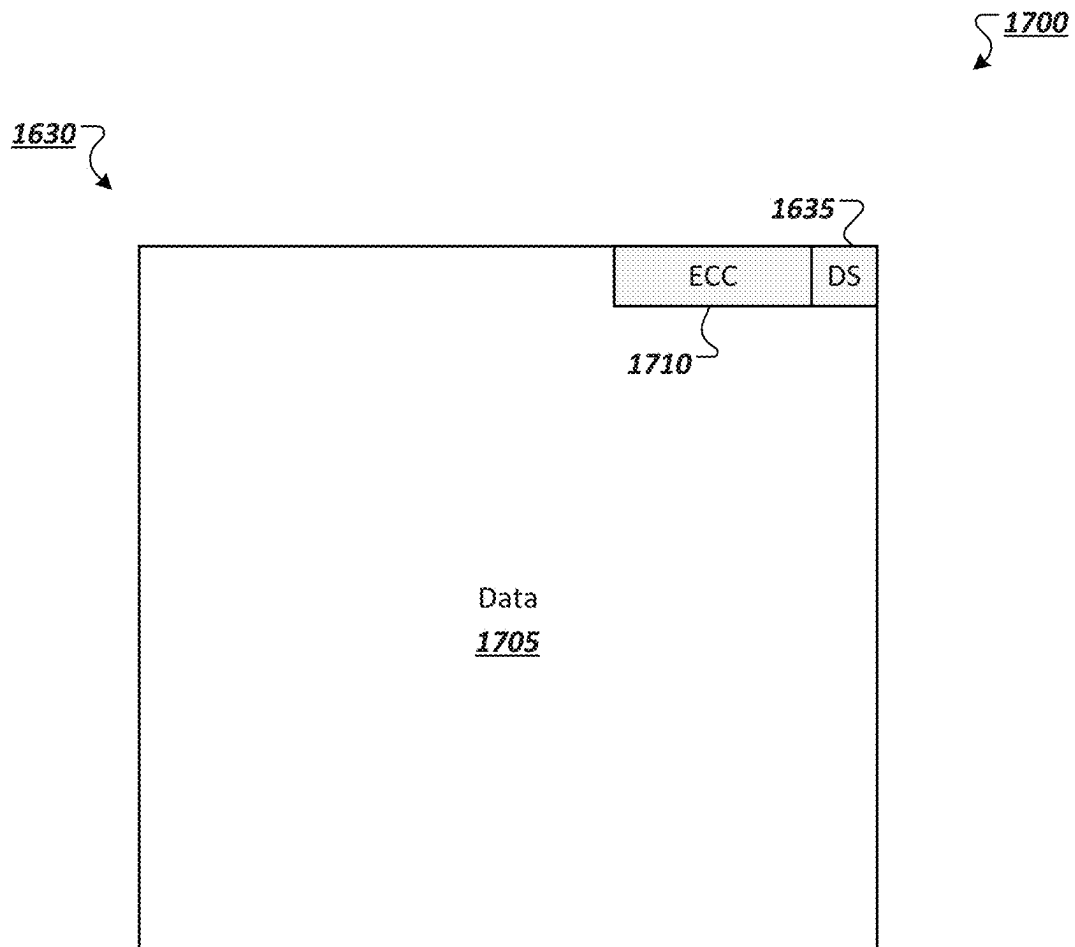


FIG. 16B



**FIG. 17**

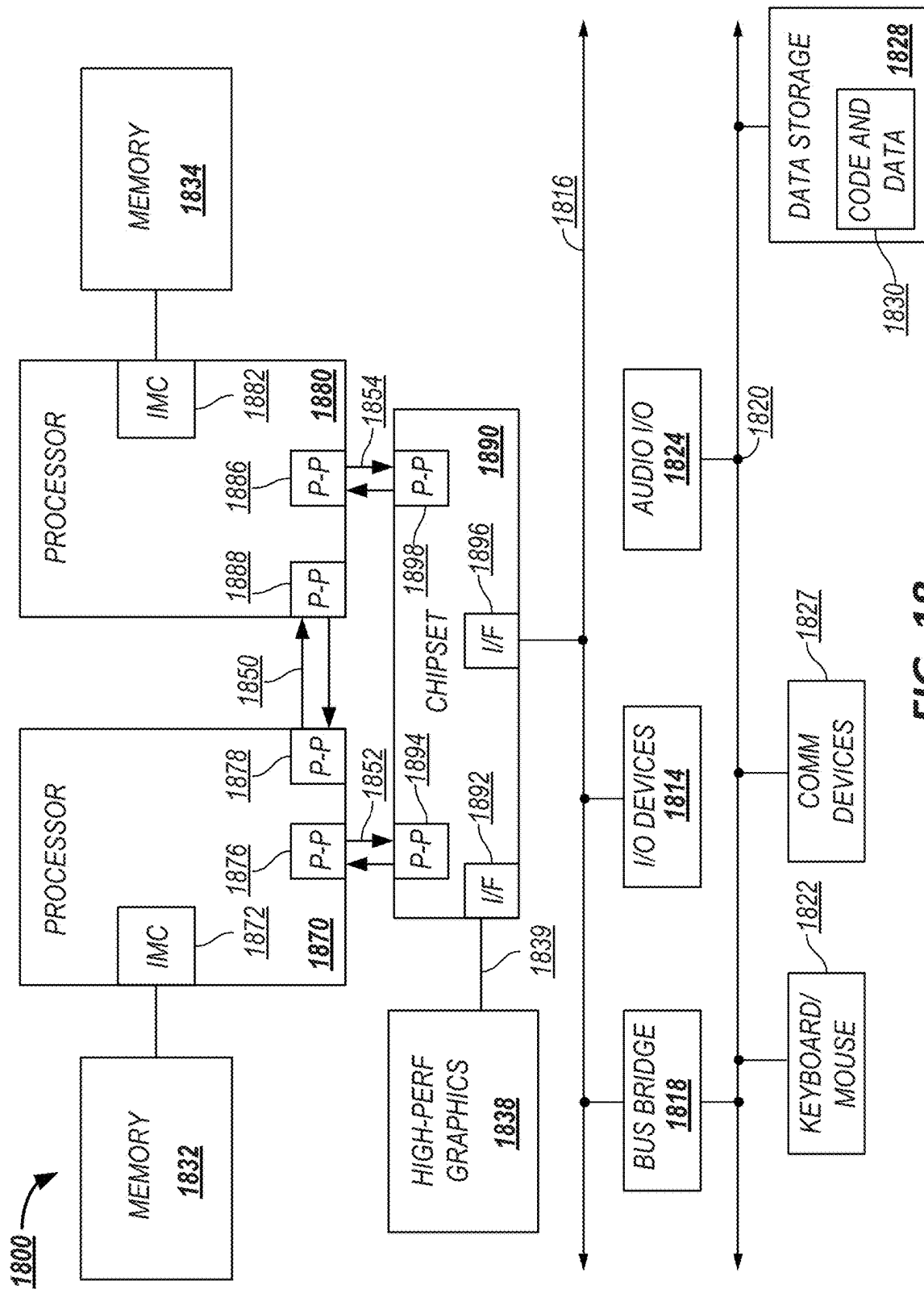
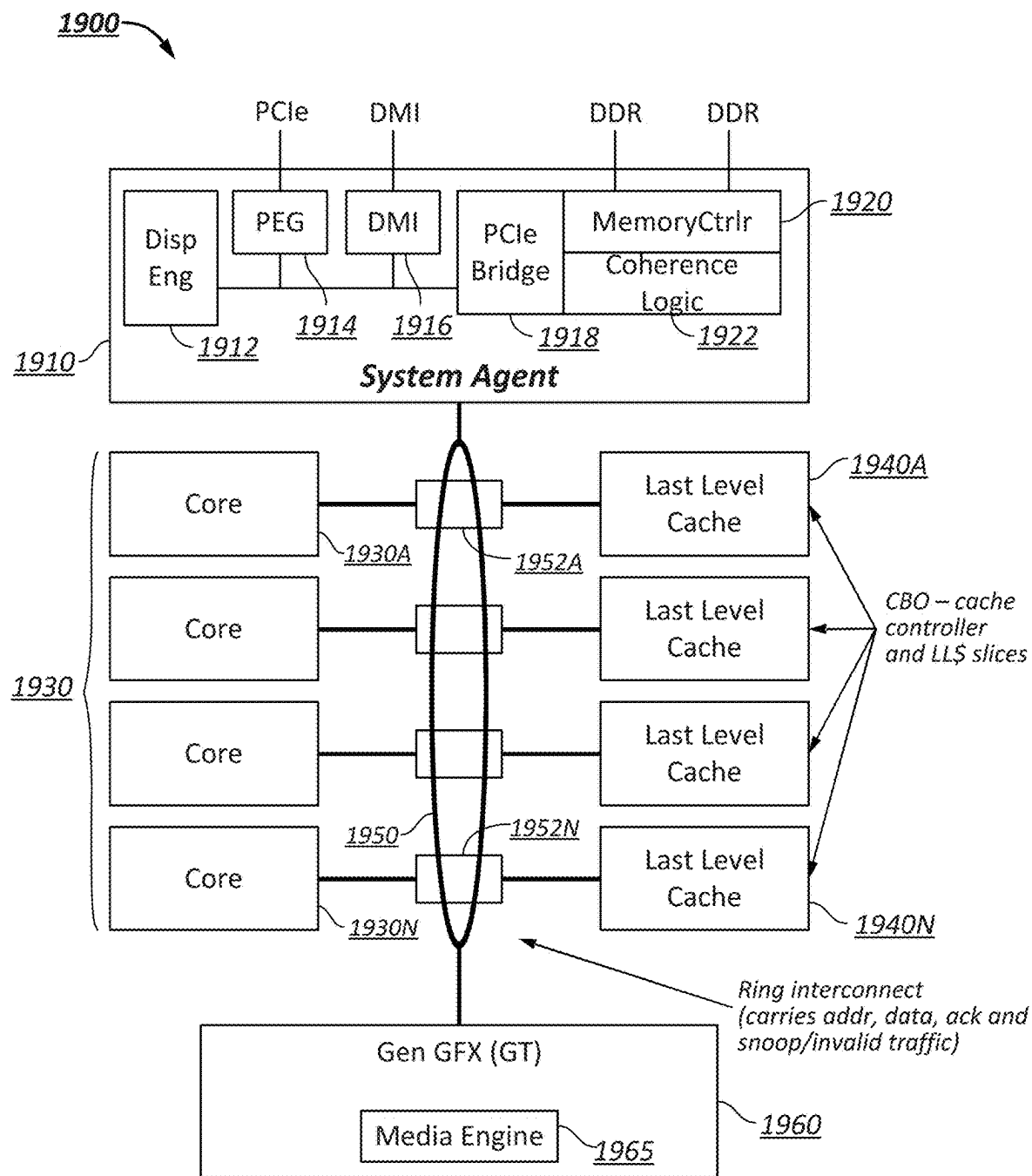


FIG. 18

**FIG. 19**

1

IMPLIED DIRECTORY STATE UPDATES**CROSS-REFERENCE TO RELATED APPLICATIONS**

This Application is a continuation (and claims the benefit of priority under 35 U.S.C. § 120) of U.S. patent application Ser. No. 16/372,247 filed on Apr. 1, 2019, entitled “IMPLIED DIRECTORY STATE UPDATES”, which application is a continuation of U.S. patent application Ser. No. 15/393,744, filed on Dec. 29, 2016, issued as U.S. Pat. No. 10,248,325 on Apr. 2, 2019, which application is a continuation of U.S. patent application Ser. No. 14/670,526, filed on Mar. 27, 2015, issued as U.S. Pat. No. 9,639,276 on May 2, 2017. The disclosures of the prior applications are considered part of (and are incorporated by reference in) the disclosure of this application.

FIELD

The present disclosure relates in general to the field of computing architectures, and more specifically, to buffered memory protocols.

BACKGROUND

Advances in semi-conductor processing and logic design have permitted an increase in the amount of logic that may be present on integrated circuit devices. As a corollary, computer system configurations have evolved from a single or multiple integrated circuits in a system to multiple cores, multiple hardware threads, and multiple logical processors present on individual integrated circuits, as well as other interfaces integrated within such processors. A processor or integrated circuit typically comprises a single physical processor die, where the processor die may include any number of cores, hardware threads, logical processors, interfaces, memory, controller hubs, etc.

As a result of the greater ability to fit more processing power in smaller packages, smaller computing devices have increased in popularity. Smartphones, tablets, ultrathin notebooks, and other user equipment have grown exponentially. However, these smaller devices are reliant on servers both for data storage and complex processing that exceeds the form factor. Consequently, the demand in the high-performance computing market (i.e. server space) has also increased. For instance, in modern servers, there is typically not only a single processor with multiple cores, but also multiple physical processors (also referred to as multiple sockets) to increase the computing power. But as the processing power grows along with the number of devices in a computing system, the communication between sockets and other devices becomes more critical.

In fact, interconnects have grown from more traditional multi-drop buses that primarily handled electrical communications to full blown interconnect architectures that facilitate fast communication. Unfortunately, as the demand for future processors to consume at even higher-rates corresponding demand is placed on the capabilities of existing interconnect architectures.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates a simplified block diagram of a system including a serial point-to-point interconnect to connect I/O devices in a computer system in accordance with one embodiment;

2

FIG. 2 illustrates a simplified block diagram of a layered protocol stack in accordance with one embodiment;

FIG. 3 illustrates an embodiment of a transaction descriptor.

FIG. 4 illustrates an embodiment of a serial point-to-point link.

FIG. 5 illustrates embodiments of potential High Performance Interconnect (HPI) system configurations.

FIG. 6 illustrates an embodiment of a layered protocol stack associated with HPI.

FIG. 7 illustrates a representation of an example state machine.

FIG. 8 illustrates a representation of an example flit sent over an example twenty-lane data link.

FIG. 9 illustrates a representation of an example flit sent over an example eight-lane data link.

FIG. 10 illustrates a representation of an example multi-slot flit.

FIG. 11 illustrates a representation of an example flit sent over an example eight-lane data link.

FIG. 12 illustrates a representation of an example multi-slot flit including a debug message.

FIG. 13 illustrates a representation of a system utilizing a buffered memory protocol.

FIG. 14 illustrates a representation of a buffer device supporting a buffered memory protocol.

FIGS. 15A-15C illustrate representations of example flits in a buffered memory protocol.

FIGS. 16A-16C illustrate example communications between a buffer device and a host device, in accordance with some implementations.

FIG. 17 illustrates a representation of an example embodiment of a cache line, in accordance with some implementations.

FIG. 18 illustrates an embodiment of a block for a computing system including multiple processor sockets.

FIG. 19 illustrates another embodiment of a block diagram for a computing system including a multicore processor.

Like reference numbers and designations in the various drawings indicate like elements.

DETAILED DESCRIPTION

In the following description, numerous specific details are set forth, such as examples of specific types of processors and system configurations, specific hardware structures, specific architectural and micro architectural details, specific register configurations, specific instruction types, specific system components, specific processor pipeline stages, specific interconnect layers, specific packet/transaction configurations, specific transaction names, specific protocol exchanges, specific link widths, specific implementations, and operation etc. in order to provide a thorough understanding of the present invention. It may be apparent, however, to one skilled in the art that these specific details need not necessarily be employed to practice the subject matter of the present disclosure. In other instances, well detailed description of known components or methods has been avoided, such as specific and alternative processor architectures, specific logic circuits/code for described algorithms, specific firmware code, low-level interconnect operation, specific logic configurations, specific manufacturing techniques and materials, specific compiler implementations, specific expression of algorithms in code, specific power down and gating techniques/logic and other

specific operational details of computer system in order to avoid unnecessarily obscuring the present disclosure.

Although the following embodiments may be described with reference to energy conservation, energy efficiency, processing efficiency, and so on in specific integrated circuits, such as in computing platforms or microprocessors, other embodiments are applicable to other types of integrated circuits and logic devices. Similar techniques and teachings of embodiments described herein may be applied to other types of circuits or semiconductor devices that may also benefit from such features. For example, the disclosed embodiments are not limited to server computer system, desktop computer systems, laptops, Ultrabooks™, but may be also used in other devices, such as handheld devices, smartphones, tablets, other thin notebooks, systems on a chip (SOC) devices, and embedded applications. Some examples of handheld devices include cellular phones, Internet protocol devices, digital cameras, personal digital assistants (PDAs), and handheld PCs. Here, similar techniques for a high-performance interconnect may be applied to increase performance (or even save power) in a low power interconnect. Embedded applications typically include a microcontroller, a digital signal processor (DSP), a system on a chip, network computers (NetPC), set-top boxes, network hubs, wide area network (WAN) switches, or any other system that can perform the functions and operations taught below. Moreover, the apparatus', methods, and systems described herein are not limited to physical computing devices, but may also relate to software optimizations for energy conservation and efficiency. As may become readily apparent in the description below, the embodiments of methods, apparatus', and systems described herein (whether in reference to hardware, firmware, software, or a combination thereof) may be considered vital to a "green technology" future balanced with performance considerations.

As computing systems are advancing, the components therein are becoming more complex. The interconnect architecture to couple and communicate between the components has also increased in complexity to ensure bandwidth demand is met for optimal component operation. Furthermore, different market segments demand different aspects of interconnect architectures to suit the respective market. For example, servers require higher performance, while the mobile ecosystem is sometimes able to sacrifice overall performance for power savings. Yet, it is a singular purpose of most fabrics to provide highest possible performance with maximum power saving. Further, a variety of different interconnects can potentially benefit from subject matter described herein.

The Peripheral Component Interconnect (PCI) Express (PCIe) interconnect fabric architecture and QuickPath Interconnect (QPI) fabric architecture, among other examples, can potentially be improved according to one or more principles described herein, among other examples. For instance, a primary goal of PCIe is to enable components and devices from different vendors to inter-operate in an open architecture, spanning multiple market segments; Clients (Desktops and Mobile), Servers (Standard and Enterprise), and Embedded and Communication devices. PCI Express is a high performance, general purpose I/O interconnect defined for a wide variety of future computing and communication platforms. Some PCI attributes, such as its usage model, load-store architecture, and software interfaces, have been maintained through its revisions, whereas previous parallel bus implementations have been replaced by a highly scalable, fully serial interface. The more recent versions of PCI Express take advantage of advances in

point-to-point interconnects, Switch-based technology, and packetized protocol to deliver new levels of performance and features. Power Management, Quality Of Service (QoS), Hot-Plug/Hot-Swap support, Data Integrity, and Error Handling are among some of the advanced features supported by PCI Express. Although the primary discussion herein is in reference to a new high-performance interconnect (HPI) architecture, aspects of the invention described herein may be applied to other interconnect architectures, such as a PCIe-compliant architecture, a QPI-compliant architecture, a MIPI compliant architecture, a high-performance architecture, or other known interconnect architecture.

Referring to FIG. 1, an embodiment of a fabric composed of point-to-point Links that interconnect a set of components is illustrated. System 100 includes processor 105 and system memory 110 coupled to controller hub 115. Processor 105 can include any processing element, such as a microprocessor, a host processor, an embedded processor, a co-processor, or other processor. Processor 105 is coupled to controller hub 115 through front-side bus (FSB) 106. In one embodiment, FSB 106 is a serial point-to-point interconnect as described below. In another embodiment, link 106 includes a serial, differential interconnect architecture that is compliant with different interconnect standard.

System memory 110 includes any memory device, such as random access memory (RAM), non-volatile (NV) memory, or other memory accessible by devices in system 100. System memory 110 is coupled to controller hub 115 through memory interface 116. Examples of a memory interface include a double-data rate (DDR) memory interface, a dual-channel DDR memory interface, and a dynamic RAM (DRAM) memory interface.

In one embodiment, controller hub 115 can include a root hub, root complex, or root controller, such as in a PCIe interconnection hierarchy. Examples of controller hub 115 include a chipset, a memory controller hub (MCH), a northbridge, an interconnect controller hub (ICH) a southbridge, and a root controller/hub. Often the term chipset refers to two physically separate controller hubs, e.g., a memory controller hub (MCH) coupled to an interconnect controller hub (ICH). Note that current systems often include the MCH integrated with processor 105, while controller 115 is to communicate with I/O devices, in a similar manner as described below. In some embodiments, peer-to-peer routing is optionally supported through root complex 115.

Here, controller hub 115 is coupled to switch/bridge 120 through serial link 119. Input/output modules 117 and 121, which may also be referred to as interfaces/ports 117 and 121, can include/implement a layered protocol stack to provide communication between controller hub 115 and switch 120. In one embodiment, multiple devices are capable of being coupled to switch 120.

Switch/bridge 120 routes packets/messages from device 125 upstream, i.e. up a hierarchy towards a root complex, to controller hub 115 and downstream, i.e. down a hierarchy away from a root controller, from processor 105 or system memory 110 to device 125. Switch 120, in one embodiment, is referred to as a logical assembly of multiple virtual PCI-to-PCI bridge devices. Device 125 includes any internal or external device or component to be coupled to an electronic system, such as an I/O device, a Network Interface Controller (NIC), an add-in card, an audio processor, a network processor, a hard-drive, a storage device, a CD/DVD ROM, a monitor, a printer, a mouse, a keyboard, a router, a portable storage device, a Firewire device, a

5

Universal Serial Bus (USB) device, a scanner, and other input/output devices. Often in the PCIe vernacular, such as device, is referred to as an endpoint. Although not specifically shown, device **125** may include a bridge (e.g., a PCIe to PCI/PCI-X bridge) to support legacy or other versions of devices or interconnect fabrics supported by such devices.

Graphics accelerator **130** can also be coupled to controller hub **115** through serial link **132**. In one embodiment, graphics accelerator **130** is coupled to an MCH, which is coupled to an ICH. Switch **120**, and accordingly I/O device **125**, is then coupled to the ICH. I/O modules **131** and **118** are also to implement a layered protocol stack to communicate between graphics accelerator **130** and controller hub **115**. Similar to the MCH discussion above, a graphics controller or the graphics accelerator **130** itself may be integrated in processor **105**.

Turning to FIG. **2** an embodiment of a layered protocol stack is illustrated. Layered protocol stack **200** can include any form of a layered communication stack, such as a QPI stack, a PCIe stack, a next generation high performance computing interconnect (HPI) stack, or other layered stack. In one embodiment, protocol stack **200** can include transaction layer **205**, link layer **210**, and physical layer **220**. An interface, such as interfaces **117**, **118**, **121**, **122**, **126**, and **131** in FIG. **1**, may be represented as communication protocol stack **200**. Representation as a communication protocol stack may also be referred to as a module or interface implementing/including a protocol stack.

Packets can be used to communicate information between components. Packets can be formed in the Transaction Layer **205** and Data Link Layer **210** to carry the information from the transmitting component to the receiving component. As the transmitted packets flow through the other layers, they are extended with additional information used to handle packets at those layers. At the receiving side the reverse process occurs and packets get transformed from their Physical Layer **220** representation to the Data Link Layer **210** representation and finally (for Transaction Layer Packets) to the form that can be processed by the Transaction Layer **205** of the receiving device.

In one embodiment, transaction layer **205** can provide an interface between a device's processing core and the interconnect architecture, such as Data Link Layer **210** and Physical Layer **220**. In this regard, a primary responsibility of the transaction layer **205** can include the assembly and disassembly of packets (i.e., transaction layer packets, or TLPs). The translation layer **205** can also manage credit-based flow control for TLPs. In some implementations, split transactions can be utilized, i.e., transactions with request and response separated by time, allowing a link to carry other traffic while the target device gathers data for the response, among other examples.

Credit-based flow control can be used to realize virtual channels and networks utilizing the interconnect fabric. In one example, a device can advertise an initial amount of credits for each of the receive buffers in Transaction Layer **205**. An external device at the opposite end of the link, such as controller hub **115** in FIG. **1**, can count the number of credits consumed by each TLP. A transaction may be transmitted if the transaction does not exceed a credit limit. Upon receiving a response an amount of credit is restored. One example of an advantage of such a credit scheme is that the latency of credit return does not affect performance, provided that the credit limit is not encountered, among other potential advantages.

In one embodiment, four transaction address spaces can include a configuration address space, a memory address

6

space, an input/output address space, and a message address space. Memory space transactions include one or more of read requests and write requests to transfer data to/from a memory-mapped location. In one embodiment, memory space transactions are capable of using two different address formats, e.g., a short address format, such as a 32-bit address, or a long address format, such as 64-bit address. Configuration space transactions can be used to access configuration space of various devices connected to the interconnect. Transactions to the configuration space can include read requests and write requests. Message space transactions (or, simply messages) can also be defined to support in-band communication between interconnect agents. Therefore, in one example embodiment, transaction layer **205** can assemble packet header/payload **206**.

Quickly referring to FIG. **3**, an example embodiment of a transaction layer packet descriptor is illustrated. In one embodiment, transaction descriptor **300** can be a mechanism for carrying transaction information. In this regard, transaction descriptor **300** supports identification of transactions in a system. Other potential uses include tracking modifications of default transaction ordering and association of transaction with channels. For instance, transaction descriptor **300** can include global identifier field **302**, attributes field **304** and channel identifier field **306**. In the illustrated example, global identifier field **302** is depicted comprising local transaction identifier field **308** and source identifier field **310**. In one embodiment, global transaction identifier **302** is unique for all outstanding requests.

According to one implementation, local transaction identifier field **308** is a field generated by a requesting agent, and can be unique for all outstanding requests that require a completion for that requesting agent. Furthermore, in this example, source identifier **310** uniquely identifies the requestor agent within an interconnect hierarchy. Accordingly, together with source ID **310**, local transaction identifier **308** field provides global identification of a transaction within a hierarchy domain.

Attributes field **304** specifies characteristics and relationships of the transaction. In this regard, attributes field **304** is potentially used to provide additional information that allows modification of the default handling of transactions. In one embodiment, attributes field **304** includes priority field **312**, reserved field **314**, ordering field **316**, and no-snoop field **318**. Here, priority sub-field **312** may be modified by an initiator to assign a priority to the transaction. Reserved attribute field **314** is left reserved for future, or vendor-defined usage. Possible usage models using priority or security attributes may be implemented using the reserved attribute field.

In this example, ordering attribute field **316** is used to supply optional information conveying the type of ordering that may modify default ordering rules. According to one example implementation, an ordering attribute of "0" denotes default ordering rules are to apply, wherein an ordering attribute of "1" denotes relaxed ordering, wherein writes can pass writes in the same direction, and read completions can pass writes in the same direction. Snoop attribute field **318** is utilized to determine if transactions are snooped. As shown, channel ID Field **306** identifies a channel that a transaction is associated with.

Returning to the discussion of FIG. **2**, a Link layer **210**, also referred to as data link layer **210**, can act as an intermediate stage between transaction layer **205** and the physical layer **220**. In one embodiment, a responsibility of the data link layer **210** is providing a reliable mechanism for exchanging Transaction Layer Packets (TLPs) between two

components on a link. One side of the Data Link Layer **210** accepts TLPs assembled by the Transaction Layer **205**, applies packet sequence identifier **211**, i.e. an identification number or packet number, calculates and applies an error detection code, i.e. CRC **212**, and submits the modified TLPs to the Physical Layer **220** for transmission across a physical to an external device.

In one example, physical layer **220** includes logical sub-block **221** and electrical sub-block **222** to physically transmit a packet to an external device. Here, logical sub-block **221** is responsible for the “digital” functions of Physical Layer **221**. In this regard, the logical sub-block can include a transmit section to prepare outgoing information for transmission by physical sub-block **222**, and a receiver section to identify and prepare received information before passing it to the Link Layer **210**.

Physical block **222** includes a transmitter and a receiver. The transmitter is supplied by logical sub-block **221** with symbols, which the transmitter serializes and transmits onto an external device. The receiver is supplied with serialized symbols from an external device and transforms the received signals into a bit-stream. The bit-stream is deserialized and supplied to logical sub-block **221**. In one example embodiment, an 8b/10b transmission code is employed, where ten-bit symbols are transmitted/received. Here, special symbols are used to frame a packet with frames **223**. In addition, in one example, the receiver also provides a symbol clock recovered from the incoming serial stream.

As stated above, although transaction layer **205**, link layer **210**, and physical layer **220** are discussed in reference to a specific embodiment of a protocol stack (such as a PCIe protocol stack), a layered protocol stack is not so limited. In fact, any layered protocol may be included/implemented and adopt features discussed herein. As an example, a port/interface that is represented as a layered protocol can include: (1) a first layer to assemble packets, i.e. a transaction layer; a second layer to sequence packets, i.e. a link layer; and a third layer to transmit the packets, i.e. a physical layer. As a specific example, a high performance interconnect layered protocol, as described herein, is utilized.

Referring next to FIG. 4, an example embodiment of a serial point to point fabric is illustrated. A serial point-to-point link can include any transmission path for transmitting serial data. In the embodiment shown, a link can include two, low-voltage, differentially driven signal pairs: a transmit pair **406/411** and a receive pair **412/407**. Accordingly, device **405** includes transmission logic **406** to transmit data to device **410** and receiving logic **407** to receive data from device **410**. In other words, two transmitting paths, i.e. paths **416** and **417**, and two receiving paths, i.e. paths **418** and **419**, are included in some implementations of a link.

A transmission path refers to any path for transmitting data, such as a transmission line, a copper line, an optical line, a wireless communication channel, an infrared communication link, or other communication path. A connection between two devices, such as device **405** and device **410**, is referred to as a link, such as link **415**. A link may support one lane—each lane representing a set of differential signal pairs (one pair for transmission, one pair for reception). To scale bandwidth, a link may aggregate multiple lanes denoted by xN, where N is any supported link width, such as 1, 2, 4, 8, 12, 16, 20, 24, 32, 64, or wider.

A differential pair can refer to two transmission paths, such as lines **416** and **417**, to transmit differential signals. As an example, when line **416** toggles from a low voltage level to a high voltage level, i.e. a rising edge, line **417** drives from

a high logic level to a low logic level, i.e. a falling edge. Differential signals potentially demonstrate better electrical characteristics, such as better signal integrity, i.e. cross-coupling, voltage overshoot/undershoot, ringing, among other example advantages. This allows for a better timing window, which enables faster transmission frequencies.

In one embodiment, a new High Performance Interconnect (HPI) is provided. HPI can include a next-generation cache-coherent, link-based interconnect. As one example, HPI may be utilized in high performance computing platforms, such as workstations or servers, including in systems where PCIe or another interconnect protocol is typically used to connect processors, accelerators, I/O devices, and the like. However, HPI is not so limited. Instead, HPI may be utilized in any of the systems or platforms described herein. Furthermore, the individual ideas developed may be applied to other interconnects and platforms, such as PCIe, MIPI, Ethernet, USB, QPI, etc.

To support multiple devices, in one example implementation, HPI can include an Instruction Set Architecture (ISA) agnostic (i.e. HPI is able to be implemented in multiple different devices). In another scenario, HPI may also be utilized to connect high performance I/O devices, not just processors or accelerators. For example, a high performance PCIe device may be coupled to HPI through an appropriate translation bridge (i.e. HPI to PCIe). Moreover, the HPI links may be utilized by many HPI based devices, such as processors, in various ways (e.g. stars, rings, meshes, etc.). FIG. 5 illustrates example implementations of multiple potential multi-socket configurations. A two-socket configuration **505**, as depicted, can include two HPI links; however, in other implementations, one HPI link may be utilized. For larger topologies, any configuration may be utilized as long as an identifier (ID) is assignable and there is some form of virtual path, among other additional or substitute features. As shown, in one example, a four socket configuration **510** has an HPI link from each processor to another. But in the eight socket implementation shown in configuration **515**, not every socket is directly connected to each other through an HPI link. However, if a virtual path or channel exists between the processors, the configuration is supported. A range of supported processors includes 2-32 in a native domain. Higher numbers of processors may be reached through use of multiple domains or other interconnects between node controllers, among other examples.

The HPI architecture includes a definition of a layered protocol architecture, including in some examples, protocol layers (coherent, non-coherent, and, optionally, other memory based protocols), a routing layer, a link layer, and a physical layer. Furthermore, HPI can further include enhancements related to power managers (such as power control units (PCUs)), design for test and debug (DFT), fault handling, registers, security, among other examples. FIG. 5 illustrates an embodiment of an example HPI layered protocol stack. In some implementations, at least some of the layers illustrated in FIG. 5 may be optional. Each layer deals with its own level of granularity or quantum of information (the protocol layer **605a,b** with packets **630**, link layer **610a,b** with flits **635**, and physical layer **605a,b** with phits **640**). Note that a packet, in some embodiments, may include partial flits, a single flit, or multiple flits based on the implementation.

As a first example, a width of a phit **640** includes a 1 to 1 mapping of link width to bits (e.g. 20 bit link width includes a phit of 20 bits, etc.). Flits may have a greater size, such as 184, 192, or 200 bits. Note that if phit **640** is 20 bits wide and the size of flit **635** is 184 bits then it takes a

fractional number of phits **640** to transmit one flit **635** (e.g. 9.2 phits at 20 bits to transmit an 184 bit flit **635** or 9.6 at 20 bits to transmit a 192 bit flit, among other examples). Note that widths of the fundamental link at the physical layer may vary. For example, the number of lanes per direction may include 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, etc. In one embodiment, link layer **610a,b** is capable of embedding multiple pieces of different transactions in a single flit, and one or multiple headers (e.g. 1, 2, 3, 4) may be embedded within the flit. In one example, HPI splits the headers into corresponding slots to enable multiple messages in the flit destined for different nodes.

Physical layer **605a,b**, in one embodiment, can be responsible for the fast transfer of information on the physical medium (electrical or optical etc.). The physical link can be point-to-point between two Link layer entities, such as layer **605a** and **605b**. The Link layer **610a,b** can abstract the Physical layer **605a,b** from the upper layers and provides the capability to reliably transfer data (as well as requests) and manage flow control between two directly connected entities. The Link Layer can also be responsible for virtualizing the physical channel into multiple virtual channels and message classes. The Protocol layer **620a,b** relies on the Link layer **610a,b** to map protocol messages into the appropriate message classes and virtual channels before handing them to the Physical layer **605a,b** for transfer across the physical links. Link layer **610a,b** may support multiple messages, such as a request, snoop, response, writeback, non-coherent data, among other examples.

The Physical layer **605a,b** (or PHY) of HPI can be implemented above the electrical layer (i.e. electrical conductors connecting two components) and below the link layer **610a,b**, as illustrated in FIG. 6. The Physical layer and corresponding logic can reside on each agent and connects the link layers on two agents (A and B) separated from each other (e.g. on devices on either side of a link). The local and remote electrical layers are connected by physical media (e.g. wires, conductors, optical, etc.). The Physical layer **605a,b**, in one embodiment, has two major phases, initialization and operation. During initialization, the connection is opaque to the link layer and signaling may involve a combination of timed states and handshake events. During operation, the connection is transparent to the link layer and signaling is at a speed, with all lanes operating together as a single link. During the operation phase, the Physical layer transports flits from agent A to agent B and from agent B to agent A. The connection is also referred to as a link and abstracts some physical aspects including media, width and speed from the link layers while exchanging flits and control/status of current configuration (e.g. width) with the link layer. The initialization phase includes minor phases e.g. Polling, Configuration. The operation phase also includes minor phases (e.g. link power management states).

In one embodiment, Link layer **610a,b** can be implemented so as to provide reliable data transfer between two protocol or routing entities. The Link layer can abstract Physical layer **605a,b** from the Protocol layer **620a,b**, and can be responsible for the flow control between two protocol agents (A, B), and provide virtual channel services to the Protocol layer (Message Classes) and Routing layer (Virtual Networks). The interface between the Protocol layer **620a,b** and the Link Layer **610a,b** can typically be at the packet level. In one embodiment, the smallest transfer unit at the Link Layer is referred to as a flit which a specified number of bits, such as 192 bits or some other denomination. The Link Layer **610a,b** relies on the Physical layer **605a,b** to frame the Physical layer's **605a,b** unit of transfer (phit) into

the Link Layer's **610a,b** unit of transfer (flit). In addition, the Link Layer **610a,b** may be logically broken into two parts, a sender and a receiver. A sender/receiver pair on one entity may be connected to a receiver/sender pair on another entity. Flow Control is often performed on both a flit and a packet basis. Error detection and correction is also potentially performed on a flit level basis.

In one embodiment, Routing layer **615a,b** can provide a flexible and distributed method to route HPI transactions from a source to a destination. The scheme is flexible since routing algorithms for multiple topologies may be specified through programmable routing tables at each router (the programming in one embodiment is performed by firmware, software, or a combination thereof). The routing functionality may be distributed; the routing may be done through a series of routing steps, with each routing step being defined through a lookup of a table at either the source, intermediate, or destination routers. The lookup at a source may be used to inject a HPI packet into the HPI fabric. The lookup at an intermediate router may be used to route an HPI packet from an input port to an output port. The lookup at a destination port may be used to target the destination HPI protocol agent. Note that the Routing layer, in some implementations, can be thin since the routing tables, and, hence the routing algorithms, are not specifically defined by specification. This allows for flexibility and a variety of usage models, including flexible platform architectural topologies to be defined by the system implementation. The Routing layer **615a,b** relies on the Link layer **610a,b** for providing the use of up to three (or more) virtual networks (VNs)—in one example, two deadlock-free VNs, VN0 and VN1 with several message classes defined in each virtual network. A shared adaptive virtual network (VNA) may be defined in the Link layer, but this adaptive network may not be exposed directly in routing concepts, since each message class and virtual network may have dedicated resources and guaranteed forward progress, among other features and examples.

In one embodiment, HPI can include a Coherence Protocol layer **620a,b** to support agents caching lines of data from memory. An agent wishing to cache memory data may use the coherence protocol to read the line of data to load into its cache. An agent wishing to modify a line of data in its cache may use the coherence protocol to acquire ownership of the line before modifying the data. After modifying a line, an agent may follow protocol requirements of keeping it in its cache until it either writes the line back to memory or includes the line in a response to an external request. Lastly, an agent may fulfill external requests to invalidate a line in its cache. The protocol ensures coherency of the data by dictating the rules all caching agents may follow. It also provides the means for agents without caches to coherently read and write memory data.

Two conditions may be enforced to support transactions utilizing the HPI Coherence Protocol. First, the protocol can maintain data consistency, as an example, on a per-address basis, among data in agents' caches and between those data and the data in memory. Informally, data consistency may refer to each valid line of data in an agent's cache representing a most up-to-date value of the data and data transmitted in a coherence protocol packet can represent the most up-to-date value of the data at the time it was sent. When no valid copy of the data exists in caches or in transmission, the protocol may ensure the most up-to-date value of the data resides in memory. Second, the protocol can provide well-defined commitment points for requests. Commitment points for reads may indicate when the data is usable; and for writes they may indicate when the written data is globally

11

observable and will be loaded by subsequent reads. The protocol may support these commitment points for both cacheable and uncacheable (UC) requests in the coherent memory space.

In some implementations, HPI can utilize an embedded clock. A clock signal can be embedded in data transmitted using the interconnect. With the clock signal embedded in the data, distinct and dedicated clock lanes can be omitted. This can be useful, for instance, as it can allow more pins of a device to be dedicated to data transfer, particularly in systems where space for pins is at a premium.

A link can be established between two agents on either side of an interconnect. An agent sending data can be a local agent and the agent receiving the data can be a remote agent. State machines can be employed by both agents to manage various aspects of the link. In one embodiment, the Physical layer data path can transmit flits from the link layer to the electrical front-end. The control path, in one implementation, includes a state machine (also referred to as a link training state machine or the similar). The state machine's actions and exits from states may depend on internal signals, timers, external signals or other information. In fact, some of the states, such as a few initialization states, may have timers to provide a timeout value to exit a state. Note that detect, in some embodiments, refers to detecting an event on both legs of a lane; but not necessarily simultaneously. However, in other embodiments, detect refers to detection of an event by an agent of reference. Debounce, as one example, refers to sustained assertion of a signal. In one embodiment, HPI supports operation in the event of non-function lanes. Here, lanes may be dropped at specific states.

States defined in the state machine can include reset states, initialization states, and operational states, among other categories and subcategories. In one example, some initialization states can have a secondary timer which is used to exit the state on a timeout (essentially an abort due to failure to make progress in the state). An abort may include updating of registers, such as status register. Some states can also have primary timer(s) which are used to time the primary functions in the state. Other states can be defined such that internal or external signals (such as handshake protocols) drive transition from the state to another state, among other examples.

A state machine may also support debug through single step, freeze on initialization abort and use of testers. Here, state exits can be postponed/held until the debug software is ready. In some instance, the exit can be postponed/held until the secondary timeout. Actions and exits, in one embodiment, can be based on exchange of training sequences. In one embodiment, the link state machine is to run in the local agent clock domain and transition from one state to the next is to coincide with a transmitter training sequence boundary. Status registers may be utilized to reflect the current state.

FIG. 7 illustrates a representation of at least a portion of a state machine used by agents in one example implementation of HPI. It should be appreciated that the states included in the state table of FIG. 7 include a non-exhaustive listing of possible states. For instance, some transitions are omitted to simplify the diagram. Also, some states may be combined, split, or omitted, while others might be added. Such states can include:

Event reset state: entered on a warm or cold reset event. Restores default values. Initialize counters (e.g., sync counters). May exit to another state, such as another reset state.

Timed reset state: timed state for in-band reset. May drive a predefined electrical ordered set (EOS) so remote receivers are capable of detecting the EOS and entering the timed reset

12

as well. Receiver has lanes holding electrical settings. May exit to an agent to calibrate reset state.

Calibrate reset state: calibration without signaling on the lane (e.g. receiver calibration state) or turning drivers off. May be a predetermined amount of time in the state based on a timer. May set an operational speed. May act as a wait state when a port is not enabled. May include minimum residency time. Receiver conditioning or staggering off may occur based on design. May exit to a receiver detect state after a timeout and/or completion of calibration.

Receiver detect state: detect presence of a receiver on lane(s). May look for receiver termination (e.g. receiver pulldown insertion). May exit to calibrate reset state upon a specified value being set or when another specified value is not set. May exit to transmitter calibrate state if a receiver is detected or a timeout is reached.

Transmitter calibrate state: for transmitter calibrations. May be a timed state allocated for transmitter calibrations. May include signaling on a lane. May continuously drive an EOS, such as an electric idle exit ordered set (or EIEIOS). May exit to compliance state when done calibrating or on expiration of a timer. May exit to transmitter detect state if a counter has expired or a secondary timeout has occurred.

Transmitter detect state: qualifies valid signaling. May be a handshake state where an agent completes actions and exits to a next state based on remote agent signaling. Receiver may qualify valid signaling from transmitter. Receiver, in one embodiment, looks for a wake detect, and if debounced on one or more lanes looks for it on the other lanes. Transmitter drives a detect signal. May exit to a polling state in response to debounce being completed for all lanes and/or a timeout or if debounce on all lanes is not complete and there is a timeout. Here, one or more monitor lanes may be kept awake to debounce a wake signal. And if debounced then the other lanes are potentially debounced. This can enable power savings in low power states.

Polling state: receiver adapts, locks on bits (e.g., by initializing a drift buffer), locks on bytes (e.g., by identifying symbol boundaries), and locks on training sequence boundaries (e.g., by locking to an electrical ordered set (EOS) and/or a training sequence header). Lanes may then be deskewed. Handshakes can also be completed to exit polling to one of several potential states. For instance, an exit can be caused (by an agent) to any one of a Link Width state, Compliance state, Loopback Marker state, or Reset state. Handshaking can include the completion of various defined receiver actions or conditions and the sending of an acknowledge message (ACK) to trigger the exit to the state corresponding to the completed set of actions and the ACK. For polling deskew, lane to lane skew at remote transmitter may be capped at a first length for top speed and a second length for slow speed. Deskew may be performed in a slow mode as well as an operational mode. Receiver may have a specific maximum to deskew lane-to-lane skew, such as 8, 16, or 32 intervals of skew. Receiver actions may also include latency fixing in some implementations. Receiver actions, in one embodiment, can be completed on successful deskew of a valid lane map. A successful handshake can be achieved, in one example, when a number of consecutive training sequence headers are received with acknowledgements and a number of training sequences with an acknowledge are transmitted after the receiver has completed its actions.

Compliance state: entered from the Polling state. An agent may be made a Compliance master or slave for validation purposes. The device (acting as master) can send a compliance pattern to another device (slave) and the slave can loop

back the pattern after re-timing it to its local clock (e.g., without undo of any polarity inversion or lane reversal). The compliance pattern can be used to characterize operation of the analog front end on some subset of lanes when Loopback is not functional or undesired. For instance, entering Loopback can be preconditioned on successful byte lock, TS lock, deskew, latency testing, and other testing reliant on the proper functioning of several piece of digital logic. With bit lock accomplished, Compliance can be entered and may be utilized for jitter or noise investigation, debug, exploring a link, among other conditioning. In other words, Compliance can serve as an alternative exit for Polling in the event the results of Polling do not allow direct entry into Loopback. The Compliance state can drive a supersequence with a transmitter from the master. Receiver looks for a wake on a monitor lane, debounces the wake, drops bad lanes, adapts, and bit locks, etc. The slave transmitter can drive the compliance pattern until its receiver actions are complete. Then loop-back is re-timed and non-deskewed. Slave receiver does similar monitor and debounce, etc. actions. Exit may be to a reset state, such as a timed reset, or to a Loopback Pattern state to start the test, among other examples. In the case of an exit to Loopback Pattern state, the master may be sent (e.g., by a software controller) to the Loopback Pattern state to try out more specialized patterns.

Agent Loopback Marker State: Loopback marker is an agent state but unlike other agent states master and slave actions and exits may be different. Loopback slave may undo any polarity inversion and/or lane reversal but may not descramble or rescrumble looped back bits. Acknowledgment exchange may not apply to slave since it is looping back. Since slave may deskew before looping back on symbol boundary, master may not be forced to re-bytelock or re-deskew but the master may re-lock training sequence to avoid locking to some alias. Means to do this may include re-seeding of LFSR, comparing TS and/or EIEOS or some combination of these. The end of the SDS marks the end of loopback setup and the start of pattern generation, checking and counting.

Agent Loopback Pattern State (or Block Link state): Entered from either a Loopback Marker state or Compliance state. In Loopback Pattern state, instead of control patterns, a master transmitter can send additional specialized patterns to supplement those looped-back in a Compliance or Loopback Pattern state. The receiver can receive specialized patterns in Loopback Pattern and check for errors in the received pattern. For transmitter adaptation both agents can be masters. For a predetermined period, the transmitter can send a pattern and a remote receiver can compare this pattern and determine a figure of merit or metric for the received pattern which is recorded in a storage element, such as a register. The comparison method and metric may be design dependent (e.g., BER with jitter injection). At the end of the period, both agents can exit to Reset for the back-channel to examine the metric and set up the next iteration of transmitter adaptation.

Link width state: agent communicates with the final lane map to remote transmitter. Receiver receives the information and decodes. Receiver may record a configured lane map in a structure after checkpoint of a previous lane map value in a second structure. Receiver may also respond with an acknowledge ("ACK"). May initiate an in-band reset. As one example, first state to initiate in-band reset. In one embodiment, exit to a next state, such as flit configuration state, is performed in response to the ACK. Further, prior to entering low power state, a reset signal may also be generated if the frequency of a wake detect signal occurrence

drops below a specified value (e.g. 1 every number of unit intervals (UIs), such as 4K UI). Receiver may hold current and previous lane maps. Transmitter may use different groups of lanes based on training sequences having different values. Lane map may not modify some status registers in some embodiments.

Flitlock configuration state: entered by a transmitter but the state is considered exited (i.e. secondary timeout moot) when both transmitter and receiver have exited to a blocking link state or other link state. Transmitter exit to a link state, in one embodiment, includes start of a data sequence (SDS) and training sequence (TS) boundary after receiving a planetary alignment signal. Here, receiver exit may be based on receiving an SDS from a remote transmitter. This state may be a bridge from agent to link state. Receiver identifies SDS. Receiver may exit to blocking link state (BLS) (or a control window) if SDS received after a descrambler is initialized. If a timeout occurs, exit may be to reset state. Transmitter drives lanes with a configuration signal. Transmitter exit may be to reset, BLS, or other states based on conditions or timeouts.

Transmitting Link State: a link state. Flits are sent to a remote agent. May be entered from a blocking link state and return to a blocking link state on an event, such as a timeout. Transmitter transmits flits. Receiver receives flits. May also exit to a low power link state. In some implementations, transmitting link state (TLS) can be referred to as the L0 state.

Blocking Link State: a link state. Transmitter and receiver are operating in a unified manner. May be a timed state during which the link layer flits are held off while the Physical layer information is communicated to the remote agent. May exit to a low power link state (or other link state based on the design). A blocking link state (BLS), in one embodiment, periodically occurs. The period is referred to as a BLS interval and may be timed, as well as may differ between slow speed and operational speed. Note that the link layer may be periodically blocked from sending flits so that a Physical layer control sequence of a length may be sent, such as during a transmitting link state or a partial width transmitting link state. In some implementations, blocking link state (BLS) can be referred to as a L0 control, or L0c, state.

Partial Width Transmitting Link State: Link state. May save power by entering a partial width state. In one embodiment asymmetric partial width refers to each direction of a two direction link having different widths, which may be supported in some designs. An example of an initiator, such as a transmitter, sending a partial width indication to enter partial width transmitting link state is shown in the example of FIG. 9. Here, a partial width indication is sent while transmitting on a link with a first width to transition the link to transmit at a second, new width. A mismatch may result in a reset. Note that speeds may not be altered but width may be. Therefore, flits are potentially sent at different widths. May be similar to a transmitting link state logically; yet, since there is a smaller width, it may take longer to transmit flits. May exit to other link states, such as a low power link state based on certain received and sent messages or an exit of the partial width transmitting link state or a link blocking state based on other events. In one embodiment, a transmitter port may turn idle lanes off in a staggered manner to provide better signal integrity (i.e. noise mitigation) as shown in the timing diagram. Here, non-retry-able flits, such as Null flits, may be utilized during periods where the link width is changing. A corresponding receiver may drop these null flits and turn idle lanes off in a staggered manner, as well

15

as record the current and previous lane maps in one or more structures. Note status and associated status register may remain unaltered. In some implementations, partial width transmitting link state can be referred to as a partial L0, or L0p, state.

Exit Partial Width Transmitting Link State: exit the partial width state. May or may not use a blocking link state in some implementations. The transmitter initiates exit, in one embodiment, by sending partial width exit patterns on the idle lanes to train and deskew them. As one example, an exit pattern start with EIEOS, which is detected and debounced to signal that the lane is ready to start the entry to a full transmitting link state, and may end with SDS or Fast Training Sequence (FTS) on idle lanes. Any failure during the exit sequence (receiver actions, such as deskew not completed prior to timeout) stops flit transfers to the link layer and asserts a reset, which is handled by resetting the link on the next blocking link state occurrence. The SDS may also initialize the scrambler/descrambler on the lanes to appropriate values.

Low Power Link State: is a lower power state. In one embodiment, it is lower power than the partial width link state, since signaling in this embodiment is stopped on all lanes and in both directions. Transmitters may use a blocking link state for requesting a low power link state. Here, receiver may decode the request and respond with an ACK or a NAK; otherwise reset may be triggered. In some implementations, low power link state can be referred to as a L1 state.

In some implementations, state transitions can be facilitated to allow states to be bypassed, for instance, when state actions of the states, such as certain calibrations and configurations, have already been completed. Previous state results and configurations of a link can be stored and reused in subsequent initializations and configurations of a link. Rather than repeating such configurations and state actions, corresponding states can be bypassed. Traditional systems implementing state bypasses, however, often implement complex designs and expensive validation escapes. Rather than using a traditional bypass, in one example, HPI can utilize short timers in certain states, such as where the state actions do not need to be repeated. This can potentially allow for more uniform and synchronized state machine transitions among other potential advantages.

In one example, a software-based controller (e.g., through an external control point for the Physical layer) can enable a short timer for one or more particular states. For instance, for a state for which actions have already been performed and stored, the state can be short-timed to facilitate a quick exit from the state to a next state. If, however, the previous state action fails or cannot be applied within the short timer duration, a state exit can be performed. Further, the controller can disable the short timer, for instance, when the state actions should be performed anew. A long, or default, timer can be set for each respective state. If configuration actions at the state cannot be completed within the long timer, a state exit can occur. The long timer can be set to a reasonable duration so as to allow completion of the state actions. The short timer, in contrast, may be considerably shorter making it, in some cases, impossible to perform the state actions without reference back to previously-performed state actions, among other examples.

In some instances, during initialization (or re-initialization) of a link, as agents progress through a state machine toward an operational link state, one or more failures or state exits can occur that cause the state to reset (e.g., to a reset or other state). In effect, the initialization of the link can loop

16

through one or more states without completing the initialization and entering a link state. In one example, a count can be maintained for the number of unproductive loops in state transitions within the initialization of a link. For instance, each time an initialization returns to a reset state without reaching a link state a counter can be incremented. The counter can be reset for the link once the link successfully enters a link state. Such counters can be maintained by agents on both sides of the link. Further, a threshold can be set, for instance, by a software-based controller utilizing one or more external control points. When the count of unproductive loops meets (or exceeds) the defined threshold initialization of the link can be suspended (e.g., set and held at or before the reset state). In some implementations, in order to recommence initialization and release the initialization from the suspended state, a software-based controller can trigger a restart or re-initialization of the link. In some instances, the software-based tools can analyze the nature of the suspended initialize and perform diagnostics, set register values, and perform other operations so as to guard against further looping of the initialization. Indeed, in some implementations, a controller can set a higher counter threshold or even override the counter, among other examples, in connection with restarting a suspended link initialization.

In some implementations of HPI, supersequences can be defined, each supersequence corresponding to a respective state or entry/exit to/from the respective state. A supersequence can include a repeating sequence of data sets and symbols. The sequences can repeat, in some instances, until completion of a state or state transition, or communication of a corresponding event, among other examples. In some instances, the repeating sequence of a supersequence can repeat according to a defined frequency, such as a defined number of unit intervals (UIs). A unit interval (UI) can correspond to the interval of time for transmitting a single bit on a lane of a link or system. In some implementations, the repeating sequence can begin with an electrically ordered set (EOS). Accordingly, an instance of the EOS can be expected to repeat in accordance with the predefined frequency. Such ordered sets can be implemented as defined 16 Byte codes that may be represented in hexadecimal format, among other examples. In one example, the EOS of a supersequence can be an EIEIOS. In one example, an EIEIOS can resemble a low frequency clock signal (e.g., a predefined number of repeating FF00 or FFF000 hexadecimal symbols, etc.). A predefined set of data can follow the EOS, such as a predefined number of training sequences or other data. Such supersequences can be utilized in state transitions including link state transitions as well as initialization, among other examples.

In some implementations of an interconnect, such as in QPI, terminations of a serial data link can be brought on and off, such as when a link is reset or initialized. This approach can introduce complexity and time into the initialization of a link. In some implementations of HPI, terminations of the link can be maintained including during a reset or re-initialization of the link. Further, HPI can permit hot-plugging of devices. When another device is introduced, either through hot-plugging or otherwise, the voltage characteristics of the lane on which the new remote agent is added will change. The local agent can sense these changes in the lane voltage to detect the presence of the remote agent and prompt initialization of the link. State machine states and timers can be defined in the state machine to coordinate the detection, configuration, and initialization of a link without terminations.

In one implementation, HPI can support re-initialization on an in-band reset without changing the termination values through the screening of a lane by the receiving agent for incoming signaling. The signaling can be used to identify good lanes. As an example, the lane can be screened for any one of a set of pre-defined signals that are to be sent by a transmitter device to facilitate discovery and configuration of the link. In one example, a supersequence can be defined corresponding to one or more initialization or re-initialization tasks. The pre-defined sequence can include an electric idle exit ordered set (EIEOS) followed by additional sequence data. In some instances, as each device on either side of a lane becomes active, the devices can begin sending a supersequence corresponding to a particular initialization state, etc. In one embodiment, two types of pin resets can be supported; power-on (or "cold") reset and warm reset. A reset initiated by software or originating (in the Physical or another layer) on one agent may be communicated in-band to the other agent. However, due to usage of an embedded clock, an in-band reset may be handled by communication to another agent using an ordered set, such as a specific electrical ordered set or EIOS.

The ordered set can be sent during initialization and a PHY control sequence (or "blocking link state") can be sent after initialization. The block link state can block the link layer from sending flits. As another example, link layer traffic may be blocked to send a few NULL flits which may be discarded at the receiver.

As introduced above, initialization, in one embodiment, can be done initially at slow speed followed by initialization at fast speed. Initialization at slow speed uses the default values for the registers and timers. Software then uses the slow speed link to setup the registers, timers and electrical parameters and clears the calibration semaphores to pave the way for fast speed initialization. As one example, initialization can consist of such states or tasks as Reset, Detect, Polling, Compliance, and Configuration, among potentially others.

In one example, a link layer blocking control sequence (i.e. a blocking link state (BLS) or L0c state) can include a timed state during which the link layer flits are held off while the PHY information is communicated to the remote agent. Here, the transmitter and receiver may start a block control sequence timer. And upon expiration of the timers, the transmitter and receiver can exit the blocking state and may take other actions, such as exit to reset, exit to a different link state (or other state), including states that allow for the sending of flits across the link.

In one embodiment, link training can be provided and include the sending of one or more of scrambled training sequences, ordered sets, and control sequences, such as in connection with a defined supersequence. A training sequence symbol may include one or more of a header, reserved portions, a target latency, a pair number, a physical lane map code reference lanes or a group of lanes, and an initialization state. In one embodiment, the header can be sent with a ACK or NAK, among other examples. As an example, training sequences may be sent as part of supersequences and may be scrambled.

In one embodiment, ordered sets and control sequences are not scrambled or staggered and are transmitted identically, simultaneously and completely on all lanes. A valid reception of an ordered set may include checking of at least a portion of the ordered set (or entire ordered set for partial ordered sets). Ordered sets may include an electrically ordered set (EOS), such as an Electrical Idle Ordered Set (EIOS) or an EIEOS. A supersequence may include a start

of a data sequence (SDS) or a Fast Training Sequence (FTS). Such sets and control supersequences can be predefined and may have any pattern or hexadecimal representation, as well as any length. For example, ordered sets and supersequences may be a length of 8 bytes, 16, bytes, or 32 bytes, etc. FTS, as an example, can additionally be utilized for fast bit lock during exit of a partial width transmitting link state. Note that the FTS definition may be per lane and may utilize a rotated version of the FTS.

Supersequences, in one embodiment, can include the insertion of an EOS, such as an EIEOS, in a training sequence stream. When signaling starts, lanes, in one implementation, power-on in a staggered manner. This may result, however, in initial supersequences being seen truncated at the receiver on some lanes. Supersequences can be repeated however over short intervals (e.g., approximately one-thousand unit intervals (or ~1KUI)). The training supersequences may additionally be used for one or more of deskew, configuration and for communicating initialization target, lane map, etc. The EIEOS can be used for one or more of transitioning a lane from inactive to active state, screening for good lanes, identifying symbol and TS boundaries, among other examples.

In one embodiment, the clock can be embedded in the data so there are no separate clock lanes. The flits sent over the lanes can be scrambled to facilitate clock recovery. The receiver clock recovery unit, as one example, can deliver sampling clocks to a receiver (i.e. the receiver recovers clock from the data and uses it to sample the incoming data). Receivers in some implementations continuously adapt to an incoming bit stream. By embedding the clock, pinout can be potentially reduced. However, embedding the clock in the in-band data can alter the manner in which in-band reset is approached. In one embodiment, a blocking link state (BLS) can be utilized after initialization. Also, electrical ordered set supersequences may be utilized during initialization to facilitate the reset (e.g., as described above), among other considerations. The embedded clock can be common between the devices on a link and the common operational clock can be set during calibration and configuration of the link. For instance, HPI links can reference a common clock with drift buffers. Such implementation can realize lower latency than elastic buffers used in non-common reference clocks, among other potential advantages. Further, the reference clock distribution segments may be matched to within specified limits.

In some implementations, HPI may support flits with a width that is, in some cases, not a multiple of the nominal lane width (e.g. using a flit width of 192 bits and 20 lanes as a purely illustrative example). Indeed, in implementations permitting partial width transmitting states, the number of lanes over which flits are transmitted can fluctuate, even during the life of the link. For example, in some instances, the flit width may be a multiple of the number of active lanes at one instant but not be a multiple of the number of active lanes at another instant (e.g., as the link changes state and lane width). In instances where the number of lanes is not a multiple of a current lane width (e.g., the example of a flit width of 192 bits on 20 lanes), in some embodiments, consecutive flits can be configured to be transmitted to overlap on lanes to thereby preserve bandwidth (e.g., transmitting five consecutive 192 bit flits overlapped on the 20 lanes).

FIG. 8 illustrates a representation of transmission of consecutive flits overlapped on a number of lanes. For instance, FIG. 8 shows a representation of five overlapping 192-bit flits sent over a 20 lane link (the lanes represented by

columns 0-19). Each cell of FIG. 8 represents a respective “nibble” or grouping of four bits (e.g., bits $4n+3:4n$) included in a flit sent over a 4UI span. For instance, a 192 bit flit can be divided into 48 four-bit nibbles. In one example, nibble 0 includes bits 0-3, nibble 1 includes bits 4-7, etc. The bits in the nibbles can be sent so as to overlap, or be interleaved (e.g., “swizzled”), such that higher-priority fields of the flit are presented earlier, error detection properties (e.g., CRC) are retained, among other considerations. Indeed, a swizzling scheme can also provide that some nibbles (and their respective bits) are sent out of order (e.g., as in the examples of FIGS. 8 and 9). In some implementations, a swizzling scheme can be dependent on the architecture of the link layer and format of the flit used in the link layer.

The bits (or nibbles) of a flit with a length that is not a multiple of the active lanes can be swizzled, such as according to the example of FIG. 8. For instance, during the first 4UI, nibbles 1, 3, 5, 7, 9, 12, 14, 17, 19, 22, 24, 27, 29, 32, 34, 37, 39, 42, 44 and 47 can be sent. Nibbles 0, 2, 4, 6, 8, 11, 13, 16, 18, 21, 23, 26, 28, 31, 33, 36, 38, 41, 43, and 46 can be sent during the next 4UI. In UIs 8-11, only eight nibbles remain of the first flit. These final nibbles (i.e., 10, 15, 20, 25, 30, 40, 45) of the first flit can be sent concurrently with the first nibbles (i.e., nibbles 2, 4, 7, 9, 12, 16, 20, 25, 30, 35, 40, 45) of the second flit, such that the first and second flits overlap or are swizzled. Using such a technique, in the present example, five complete flits can be sent in 48UI, with each flit sent over a fractional 9.6 UI period.

In some instances, swizzling can result in periodic “clean” flit boundaries. For instance, in the example of FIG. 8, the starting 5-flit boundary (the top line of the first flit) may also be referred to as a clean flit boundary since all lanes are transmitting starting nibble from same flit. Agent link layer logic can be configured to identify swizzling of lanes and can reconstruct the flit from the swizzled bits. Additionally, physical layer logic can include functionality for identifying when and how to swizzle a stream of flit data based on the number of lanes being used at the moment. Indeed, in a transition from one link width state to another, agents can configure themselves to identify how swizzling of the data stream will be employed.

As noted above, links can transition between lane widths, in some instances operating at an original, or full, width and later transitioning to (and from) a partial width utilizing fewer lanes. In some instances, the defined width of a flit may be divisible by the number of lanes. For instance, the example of FIG. 9 illustrates such an example, where the 192-bit flit of the previous examples is transmitted over an 8-lane link. As represented in FIG. 9, 4-bit nibbles of a 192-bit flit can be evenly distributed and transmitted over 8 lanes (i.e., as 192 is a multiple of 8). Indeed, a single flit may be sent over 24 UI when operating at an 8-lane partial width. Further, each flit boundary can be clean in the example of FIG. 9. While clean flit boundaries can simplify the state transitions, determinism, and other features, allowing for swizzling and occasional jagged flit boundaries can allow for the minimization of wasted bandwidth on a link.

Additionally, while the example of FIG. 9, shows lanes 0-7 as the lanes that remained active in a partial width state, any set of 8 lanes can potentially be used. Note also that the examples above are for purposes of illustration only. The flits can potentially be defined to have any width. Links can also have potentially any link width. Further, the swizzling scheme of a system can be flexibly constructed according to the formats and fields of the flit, the preferred lane widths in a system, among other considerations and examples.

The operation of the HPI PHY logical layer can be independent of the underlying transmission media provided the latency does not result in latency fixing errors or timeouts at the link layer, among other considerations.

Link Layer

The Link layer can abstract the Physical layer from the Protocol layer, handle flow control between two protocol agents, and provide virtual channel services to the Protocol layer (Message Classes) and Routing layer (Virtual Networks). In some implementations, the Link layer can deal with a fixed quantum of information, termed a flit. In one example, the flit can be defined to be 192 bits in length. However, any range of bits, such as 81-256 (or more) may be utilized in different variations. A large flit size, such as 192 bits, may include format, cyclic redundancy check (CRC), error correction code (ECC), and other features. For instance, a larger flit length can also permit the CRC field to be expanded (e.g., to 16 bits) to handle the larger flit payload. The number of phits or unit intervals (UI) (e.g., the time used to transfer a single bit or phit, etc.) to transfer a single flit can vary with link width. For instance, a 20 lane or bit link width can potentially transfer a single 192 bit flit in 9.6 UI, while an 8 lane link width transfers the same flit in 24 UI, among other potential examples. The link layer crediting and protocol packetizing can also be based on a flit.

FIG. 10 illustrates a representation 1000 of a generalized flit for an 8 lane link width. Each column of the representation 1000 can symbolize a link lane and each row a respective UI. In some implementations, a single flit can be subdivided into two or more slots. Distinct messages or link layer headers can be included in each slot, allowing multiple distinct, and in some cases, independent messages corresponding to potentially different transactions to be sent in a single flit. Further, the multiple messages included in slots of a single flit may also be destined to different destination nodes, among other examples. For instance, the example of FIG. 10 illustrates a flit format with three slots. The shaded portions can represent the portion of the flit included in a respective slot.

In the example of FIG. 10, three slots, Slots 0, 1, and 2, are provided. Slot 0 can be provided 72 bits of flit space, of which 22 bits are dedicated to message header fields and 50 bits to message payload space. Slot 1 can be provided with 70 bits of flit space, of which 20 bits are dedicated to message header fields and 50 bits to message payload space. The difference in message header field space between can be optimized to provide that certain message types will be designated for inclusion in Slot 0 (e.g., where more message header encoding is utilized). A third slot, Slot 2, can be provided that occupies substantially less space than Slots 0 and 1, in this case utilizing 18 bits of flit space. Slot 2 can be optimized to handle those messages, such as acknowledgments, credit returns, and the like that do not utilize larger message payloads. Additionally, a floating payload field can be provided that allows an additional 11 bits to be alternatively applied to supplement the payload field of either Slot 0 or Slot 1.

Continuing with the specific example of FIG. 10, other fields can be global to a flit (i.e., apply across the flit and not to a particular slot). For instance, a header bit can be provided together with a 4-bit flit control field that can be used to designate such information as a virtual network of the flit, identify how the flit is to be encoded, among other examples. Additionally, error control functionality can be provided, such as through a 16-bit cyclic CRC field, among other potential examples.

A flit format can be defined so as to optimize throughput of messages on the Link layer. Some traditional protocols have utilized unslotted, smaller flits. For instance, in QPI an 80-bit flit was utilized. While the flit throughput of a larger (e.g., 192-bit flit) may be lower, message or packet throughput can be increased by optimizing use of the flit data. For instance, in some implementations of QPI, the entire 80-bit flit space was utilized regardless of the message size or type. By subdividing a larger flit into slots of predetermined lengths and fields, the 192 flit length can be optimized realizing higher efficiency even in instances when one or more of the available slots are sometimes unused. Indeed, Link layer traffic can be assumed to include many different types of messages and traffic, including messages and packets with varying header lengths and fields. The respective lengths and organization of slots defined in a flit can be defined so as to correspond with the statistical or expected frequency of various messages and the needs of these messages. For instance, two larger slots can be defined for every small slot, to accommodate an expected statistical frequency of messaging using these larger message types and header lengths, among other example. Further, flexibility can also be provided to further accommodate the varied traffic, such as through a floating payload field, as in the example of FIG. 10. In some instances, a flit format can be fixed, including the bits dedicated to particular slots in the flit.

In the example of FIG. 10, a “Hdr” field can be provided for the flit generally and represent a header indication for the flit. In some instances, the Hdr field can indicate whether the flit is a header flit or a data flit. In data flits, the flit can still remain slotted, but omit or replace the use of certain fields with payload data. In some cases, data fields may include an opcode and payload data. In the case of header flits, a variety of header fields can be provided. In the example of FIG. 10, “Oc” fields can be provided for each slot, the Oc field representing an opcode. Similarly, one or more slots can have a corresponding “msg” field representing a message type of the corresponding packet to be included in the slot, provided the slot is designed to handle such packet types, etc. “DNID” fields can represent a Destination Node ID, a “TID” field can represent a transaction or tracker ID, a “RHTID” field can represent either a requestor node ID or a home tracker ID, among other potential fields. Further, one or more slots can be provided with payload fields. Additionally, a CRC field can be included within a flit to provide a CRC value for the flit, among other examples.

In some implementations, link width can vary during the life of the link. For instance, the Physical layer can transition between link width states, such as to and from a full or original lane width and a different or partial lane width. For example, in some implementations, a link can be initialized to transfer data over 20 lanes. Later, the link can transition to a partial width transmitting state where only 8 lanes are actively used, among many other potential examples. Such lane width transitions can be utilized, for instance, in connection with power management tasks governed by one or more power control units (PCU) among other examples.

As noted above, link width can influence flit throughput rate. FIG. 11 is a representation of an example 192-bit flit sent over an 8 lane link, resulting in throughput of the flit at 24 UI. Further, as shown in the example of FIG. 11, bits of the flit can be sent out of order in some instances, for example, to send more time-sensitive fields earlier in the transfer (e.g., flit type fields (e.g., data or header flit), opcodes, etc.), preserve or facilitate particular error detection or other functionality embodied in the flit, among other

examples. For instance, in the example of FIG. 11, bits 191, 167, 143, 119, 95, 71, 47, and 23 are sent in parallel on lanes L7 through L0 during a first UI (i.e., UI0) of transfer, while bits 168, 144, 120, 96, 72, 48, 24, and 0 are sent during the 24th (or final) UI of the flit transfer (i.e., UI23). It should be appreciated that other ordering schemes, flit lengths, lane widths, etc. can be utilized in other implementations and examples.

In some instances, the length of the flit can be a multiple of the number of active lanes. In such instances, the flit can be transmitted evenly on all active lanes and transfer of the flit can end substantially simultaneously at a clean (i.e., non-overlapping) boundary. For example, as shown in the representation of FIG. 8, bits of a flit can be considered to be transmitted in consecutive groupings of 4 bits, or “nibbles.” In this example, a 192 bit flit is to be transferred over an 8 lane link. As 192 is a multiple of 8, the entire flit can be cleanly transferred over the 8 lane link in 24 UI. In other instances, the flit width may not be a multiple of the number of active lanes. For instance, FIG. 9 shows another representation of an example 192 bit transferred over 20 lanes. As 192 is not evenly divisible by 20, transfer of the full flit would require a non-integer number of intervals (e.g., 9.6 UI). In such cases, rather than wasting “extra” lanes not utilized during the 10th UI of transfer, a second overlapping flit can be transferred with the final bits of a preceding flit. Such overlapping, or swizzling, of the flits can result in jagged flit boundaries and flit bits sent out of order in some implementations. The pattern utilized for the transfer can be configured to allow more time-sensitive fields of the flit to be transferred earlier in the flit, preservation of error detection and correction, among other considerations. Logic can be provided in one or both of the Physical and Link layers to transfer flit bits according to such patterns and dynamically change between patterns based on the current link width. Further logic can be provided to re-order and re-construct flits from such swizzled or ordered bit streams, among other examples.

In some implementations, flits can be characterized as header flits (e.g., bearing packet header data) or data flits (e.g., bearing packet payload data). Returning to FIG. 10, a flit format can be defined that includes three (3) distinct slots (e.g., 0, 1, and 2), allowing up to three headers to be transferred in a single flit (e.g., one header in each slot). In the example of FIG. 10, three slots, Slots 0, 1, and 2, are provided. Slot 0 can be provided 72 bits of flit space, of which 22 bits are dedicated to message header fields and 50 bits to message payload space. Slot 1 can be provided with 70 bits of flit space, of which 20 bits are dedicated to message header fields and 50 bits to message payload space. The difference in message header field space between can be optimized to provide that certain message types will be designated for inclusion in Slot 0 (e.g., where more message header encoding is utilized). A third slot, Slot 2, can be provided that occupies substantially less space than Slots 0 and 1, in this case utilizing 18 bits of flit space. Slot 2 can be optimized to handle those messages, such as acknowledgments, credit returns, and the like that do not utilize larger message payloads. Additionally, a floating payload field can be provided that allows an additional 11 bits to be alternatively applied to supplement the payload field of either Slot 0 or Slot 1.

In some implementations, by allowing a field to float between the two slots, extra bits can be provided as needed for certain messages while still staying within a predefined flit length (e.g., 192 bits) and maximizing the utilization of the bandwidth. Turning to the examples of FIG. 19, two

instances **1905**, **1910** of an example 192-bit flit are shown on an 8 lane data link. In one instance, a flit (e.g., **1905**) can include three slots, Slots **0**, **1**, and **2**. Each of Slots **0** and **1** can include 50-bit payload fields. The floating field can be provided to alternatively extend the payload field of the either Slot **0** or Slot **1** by the field length (e.g., 11 bits) of the floating field. The use of a floating field can further extend the efficiency gains provided through a defined, multi-slot flit format. The sizing of the slots within the flit, and the types of messages that can be placed in each slot, can potentially provide increased bandwidth even with a reduced flit rate.

In the particular example of FIG. **10**, the messages that can use Slots **1** and **2** can be optimized, reducing the number of bits to be set aside to encode these slots' opcodes. When a header having more bits that Slot **0** can provide enters the Link layer, slotting algorithms can be provided to allow it to take over Slot **1** payload bits for additional space. Special control (e.g. LLCTRL) flits may also be provided that consume all three slots worth of bits for their needs. Slotting algorithms may also exist to allow individual slots to be utilized while other slots carry no information, for cases where the link is partially busy.

In one embodiment, the Link Layer can additionally define special control flits that may be used, for instance, for debug messages and other uses. In one example, LLCTRL-DEBUG flits may be sent by the Link Layer when an enable debug control field is set. When this bit is not set, LLCTRL-DEBUG flits may not be transmitted on the link. Debug Packets may be important to expose internal states of devices connected by HPI that are otherwise inaccessible. The contents of debug packets can also be implementation specific. Contents could include things like branch info (source and target IPs), time-stamps, indication of an internal event trigger, and so on. The exposed data can be, for instance, by monitoring devices such as logic analyzers for post-processing and failure analysis. An example flit encoding of a Debug message type is illustrated in FIG. **12**.

In some implementations, principles and protocols of a general purpose I/O interconnect (GPIO) architecture, such as HPI, can be utilized to implement a buffered memory interface and corresponding protocol. For instance, the physical and link layer definitions outlined above can also be implemented in a buffered memory protocol. Indeed, logic used to support the physical and link layers of the GPIO protocol can be reused at interfaces supporting the buffered memory protocol. The buffered memory protocol can also share message classes, such as a request, response, and writeback message class, among other examples. While opcode values within the buffered memory protocol message can be interpreted differently than in HPI (or another GPIO protocol), the same general packet formats can be utilized in both the buffered memory protocol and the GPIO interconnect upon which it is built. Indeed, additional packet and flit fields can be unique to both the buffered memory protocol and GPIO interconnect protocol.

In some implementations, a buffered memory protocol utilizing the physical and link layers of HPI can be a transactional interface and protocol. This can permit data to be returned out of order to a series of received requests (e.g., read requests). The buffered memory interface can be used to interconnect a buffer chip to a CPU in some implementations. In one implementation, While some protocols send DRAM commands such as Activate, Read, Write, and Refresh across a buffered memory bus, in some implementations of the buffered memory protocol discussed herein, a read or write command can simply be sent with an address.

The buffer chip can then decode the specific rank and bank, and breaks the request down to DRAM commands. Speculative and demand reads can also be supported.

Turning to FIG. **13**, a simplified block diagram **1300** is shown illustrating an example topology of a computing system including CPU devices **1305**, **1310** interconnected by an HPI-compliant link. Each CPU **1305**, **1310** can be likewise connected to one or more respective buffer devices **1315a-1** using corresponding buffered memory protocol links ("MemLink"). As noted above, in some implementations, the buffered memory protocol interconnect can be based on the GPIO protocol, in that the physical and link layers of the buffered memory protocols are based on the same physical and link layer definitions of the GPIO protocol (e.g., HPI). Although not illustrated in FIG. **13**, the CPUs **1305**, **1310** can be further connected to one or more downstream devices using the GPIO protocol.

As further shown in the example of FIG. **13**, buffer devices **1315a-1** can be connected to memory devices, such as dual in-line memory module (DIMM) devices. The memory corresponding to each buffer device can be considered local to the CPU (e.g., **1305**, **1301**) to which the buffer device is connected. However, other devices (including the other CPU) can access the memory by other sockets using GPIO protocol-compliant links. In some implementations, a port running the buffered memory protocol may only support the commands for communicating with the memory and only support the buffered memory protocol (i.e., not the GPIO protocol and the buffered memory protocol). Additionally, in some implementations, the GPIO interconnect protocol may support routing and indicate such information (e.g., in its packets) such as the requesting and destination node identifiers. The buffered memory protocol, on the other hand, may be a point-to-point interface that does not utilize routing. Consequently, some fields used in the GPIO protocol may be dispensed with in packets sent using the buffered memory interfaces. Instead, fields can be designated for use in carrying address decode information host to buffer, among other examples.

In further implementations, buffer devices **1315a-1** can support a two level memory topology with some amount of fast memory (e.g., DRAM) serving as a cache for a larger, slower memory (e.g., non-volatile memory). In one such implementation, one or more of the buffer devices **1315a-1** can use DDR as near, fast memory and transactional DDR DIMMs as the larger "far" memory, among other examples. Transactional DIMMs can utilize protocols (e.g., DDR-Transactional (DDR-T)) to communicate to a volatile memory single in-line memory module (SIMM) using transactional commands.

Turning to FIG. **14**, a simplified block diagram **1400** is presented showing a more detailed representation of a buffer chip **1315** connected to near and far memory modules (e.g., **1405**, **1410**). As noted in the example above, the buffer chip **1315** can be communicatively coupled to a CPU device or other processing device **1305** through a buffered memory protocol interconnect link **1415** utilizing a defined buffered memory protocol. The buffer chip **1315** can include logic to perform additional functions. For instance, an implementation of the buffer chip **1315** can include a memory controller **1420**, a scheduler **1425**, flow control logic **1430**, error detection logic **1435**, and directory state management logic **1436**. Logic can be implemented using hardware circuitry, firmware, and/or software.

In some implementations, the memory controller **1420** can include logic to convert requests into the memory specific protocol (e.g., DDR4) of its memory devices. The

25

memory controller **1420** can abstract the details of these memory-specific protocols from the host controller logic **1440** of the CPU **1305** (or another host) which communicates with the buffer chip **1315** using link **1415**. Scheduler **1425** can include logic to reorder and arbitrate responses to requests in order to achieve higher performance. Buffer chip **1315** can additionally offer features such as flow control and error detection. In some implementations, flow control logic **1430** can be embodied in scheduler **1425** or otherwise interoperate with scheduler **1425** to ensure more efficiency and higher performance of the buffer chip **1315**. Error detection logic **1435** can include logic to support an error correction code (ECC) detection scheme as well as logic for initiating correction or handling of detected errors. Directory state management logic **1436** can maintain the directory state of each line in memory. In response to some transactions, the directory state of a corresponding line can change. The directory state management logic **1436** can update records identifying the current directory state of each line in memory. In some cases, the directory state management logic **1436** can receive data from a home agent (e.g., at host **1305**) to indicate a change to the directory state of a line of memory. In other instances, directory state management logic **1436** can change include logic to automatically update the directory state (i.e., without the host indicating the change) based on the nature of a corresponding request. This can save bandwidth, among other example advantages, as bandwidth used to indicate at least a portion of the directory state changes can be preserved, with the directory state management logic **1436** allowing a portion of these directory state changes to be identified directly at the memory controller (e.g., **1420** at **1315**).

In some implementations, CPU **1305** can include host controller logic **1440** that can translate CPU (or host) addresses to memory addresses, including permuting across channels and sockets, among other functionality. The host **1305** can include multiple processing cores **1445a-d** that can allow multiple processes to execute in parallel on the host **1305**. Additionally, the host can make use of one or more caches to cache previously accessed lines from memory (e.g., **1405**, **1410**) such that the host **1305** can re-accesses these lines without re-requesting the data from the buffer chip **1315**. This can assist with addressing higher latency introduced through the buffer chip **1315** in a buffered memory architecture. Caches can include local and intermediate caches, as well as last level cache (LLC) **1450a-d**. Cache, including LLC, can be shared between multiple cores **1445a-d**. Indeed, a variety of caching architectures can be utilized with hosts employing one or multiple processing cores.

As noted above, the link layer of a buffered memory protocol can be based on that of a GPIO protocol. For instance, the 192-bit, 3-slot flit of an HPI protocol (e.g., described and shown in FIG. **10**) can be utilized in a buffered memory protocol. A packet in the buffered memory protocol can embody an entire command. A packet can be implemented as one or more flits. FIGS. **15A-15C** can represent implementations of a flit in a buffered memory protocol. In some implementations, read data can be sent back from the buffer device to a host via a combination of headered and headerless memory data (MemData) packets. An entire cache line can be transferred in three headerless flits (rather than five, as with headered flits). A headerless packet can thereby provide for higher throughput on what may be the most constrained portion of the interface. In order to fit into three flits, the headerless packet removes a number of fields and makes assumptions as to their values. The assumptions

26

made may be true for most, but all read returns. If any assumptions are not true, the headered packet is to be used, which contains all fields. In some implementations, a headered packet can begin a sequence of read returns to permit early identification of a tracker identifier (e.g., HTID, RTID, etc.) for use in mapping a read return to a read request in a transactional buffered memory protocol.

In some implementations, headerless packets can contain the information outlined in Table 1:

TABLE 1

Headerless Packet Fields		
Field	Qty	Description
Data	512	64 byte cache line data
Directory	2	Two directory bits, which are stored with the data, generally as part of the ECC bits
HTID	11	Request transaction ID. Due to early HTID, the packet contains the HTID for the next read data packet
CRC	48	16 bits per of CRC per flit
HIB	3	Header indication bit. One per flit. This bit is always 0 for each flit of headerless packets.

Further, FIGS. **15A-15C** illustrate a sequence of three flits (e.g., flit 0 (at FIG. **15A**), flit 1 (at FIG. **15B**), and flit 2 (at FIG. **15C**)) that can be used to transmit an entire cache line in a read return. Table 2 provides a summary of an example of the three flits utilized in a headerless packet:

TABLE 2

Headerless Packet	
Flit	Contents
Data flit 0	Data from the first 32 bytes of the cache line.
Data flit 1	Remainder of the first 32 bytes, plus the first part of the second 32 bytes.
Data flit 2	Remainder of the second 32 bytes

An HIB bit can be set to indicate whether the packet is headered or headerless. HIB may be set to indicate a headerless packet only for data flits following a header flit. This allows certain other packets to be interleaved in the middle of a packet. When the interleaved packet is completed, the data portions of the original headered packet can be sent with HIB=0 in the flits. This technique can also be used to delay the data packets by inserting null flits. In some implementations, the start of a headerless packet is signaled by the HIB bit being a 0 when normally a header would be required. All other packets, including null flits, may contain a header with the HIB bit set.

As introduced above, headered packets may be a total of five flits in length, and may include one header flit and 4 data flits. In order to reduce idle latency, the headered flit may send data flit 1 twice. The first time it is sent with only the data from the first 32 bytes and is referred to as flit 1a. This allows the flit to be sent before the second 32 bytes of data is available from the memory devices. The result is that the critical 32 bytes chunk of data arrives earlier at the host for the idle case. Flit 1 can then be sent a second time, this time complete. It will contain a repeat of the first 32 byte data portion plus the second 32 byte portion. Table 3 describes the five flits of a headered packet:

TABLE 3

Headered Packet	
Flit	Contents
Header	Packet Header
Data flit 0	Data from the first 32 bytes of the cache line
Data flit 1a	Remainder of the first 32 bytes only. Second 32-byte portion is reserved. The other flit payload bits (RTID[6:3]) are identical to Data Flit 1.
Data flit 1	Complete Flit 1. First 32-byte portion is resent. Second 32-byte portion also included.
Data flit 2	Remainder of the second 32 bytes.

In some implementations, headered MemData packets may not be cancelled and are to be followed by the data flits. A header is not to be sent until it known that the data can also be sent. The full cache line may not be accumulated in the buffer before the header is sent.

In traditional systems, home agents on various hosts can be tasked with explicitly updating directory state of corresponding lines stored in memory after each read. Explicitly, updating directory state can involve writing back the full cache line plus the directory state. Such an implementation can result in significant amounts of bandwidth being used to update the directory state, particularly in buffered memory architectures where a home agent and memory controller are not tightly coupled. A buffered memory link can be used to interconnect the home agent and memory controller. This buffered memory link can thus be used by the memory controller both to receive requests of a host as well as receive writebacks whose sole purpose, in some cases, may be to simply designate a change to a directory state of a given line based on a read. Further, in cases where the directory state is indicated within the cache line itself, directory state changes can involve transmitting entire copies of the cache line from the home agent to the memory controller over the buffered memory link, resulting in high bandwidth costs for such updates.

These are other issues can be addressed using an improved architecture that utilizes new command types allowing the memory controller to explicitly or implicitly direct the memory controller to update directory state when possible, without direction from the home agent or other logic outside the memory controller. Significant write bandwidth can thus be save on the interface between home agents and memory controllers. This interface may be buffered memory link interface. Other architectures can also utilize such a model, such as an internal mesh where memory controllers are implemented on the same system on chip (SoC) as the home agent, among other examples.

FIGS. 16A-16C are simplified diagrams 1600a-c illustrating example read transactions over a buffered memory interconnect involving a host 1305 and buffer chip 1315. While this example specifically utilizes a read transaction, it should be appreciated that the principles illustrated in the examples of FIGS. 16A-16C can also be applied to other types of requests including memory invalidate requests. Returning to the illustration in FIGS. 16A-16C, a memory read 1610 can be sent from the host 1305 to the buffer chip 1315 over the buffered memory interconnect. The memory read, or read request 1610, can originate at the local host 1305 or from another remote host to which the local host 1305 is connected (e.g., via a HPI link, among other examples). The memory controller of the buffer chip 1315 can receive the read request 1610 and determine a corre-

sponding memory read operation 1615 to perform to retrieve a corresponding line from memory 1605 in response to the request.

The read request 1610 can be a request according to a buffered memory protocol. The buffered memory protocol read request 1610 can correspond to a read request 1650 of a general purpose input/output (GPIO) interconnect protocol. The GPIO protocol can also support several different request types, including read and memory invalidate requests. In one example, GPIO protocol request types can include:

RdCur: to request an uncacheable “snapshot” copy of a particular cache line

RdCode: request an shared (or “S”) copy of the cache line

RdData: request an exclusive (or “E”) or S copy of the cache line

RdDataMig: request a modified (or “M”), E, or S copy of the cache line

RdInvOwn: request an M or E copy of the cache line

RdInv: request an E copy of the cache line from the home agent; any modified copy is committed to memory before receiving the data

InvItoE: request ownership of a cache line in E state, without a data response

InvXtoI: invalidate all caches, including requesting agent’s

InvItoM: request ownership of cache line in M-empty state, without a data response, with intent of writing back modified line soon afterward

NonSnapRd: request an uncacheable “snapshot” copy of cache line from memory

As noted in the above example, a set of coherency states can be maintained for the GPIO protocol, including shared (“S”), exclusive (“E”), modified (“M”), and invalid (“I”) states, among other potential examples.

The buffered memory protocol can, itself, define multiple different request types. The buffered memory protocol can also maintain directory states for each cache line, including any (“A”) (e.g., any one of the M, E, S, or I coherency states), shared (“S”) (e.g., any one of the S or I coherency states), exclusive (“E”), invalid (“I”) (e.g., guaranteed to be in I coherency state), or unknown, among other examples. In one example, the read request types of the buffered memory protocol can include:

MemRd: normal memory read. Directory information is left unmodified

MemRdXtoI: Memory read, Directory result is Invalid. If the directory indicates anything other than I, it is written back as I

MemInvXtoI: Memory Invalidate. Directory result is Invalid. No read data is returned. The memory controller reads the cache line, and rewrites it with the directory set to I if it was not already that way.

MemRdXtoS: Memory Read, Directory result is Shared. If the directory indicates anything other than S, it is written back as S.

MemRdData: Memory Read, rewrite with directory (if necessary) as follows: I to A; S has no change; A has no change, if a change is required from A, the host must specifically change it

MemRdXtoA: Memory read, Directory result is Any. If the directory indicates anything other than A, it is written back as A.

MemInvXtoA: Memory Invalidate, Directory result is Any. No read data is returned. The memory controller reads the cache line, and rewrites it with the directory set to A, if it was not already that way.

Buffered memory protocol read request types can correspond to GPIO protocol read request types. A buffer, or memory controller, can sometimes cover only a sub-set of caching agents, so the home logic can be responsible for issuing the right memory command depending if the requesting caching agent is covered by the directory or not. In one implementation, caching agents can be grouped between local and remote caching agents (such as in the examples of Tables 4 and 5), but other alternate groupings can be utilized in other implementations. In the examples of Tables 4 and 5, “local” requesters are agents not covered by the directory, and “remote” caching agents are covered by the directory. Tables 4 and 5 outline associations between GPIO protocol read request types and buffered memory (MEM) protocol request types. Table 4 outlines read requests from a “local” host or caching agent. Table 5 outlines read requests originating from “remote” hosts or caching agents.

TABLE 4

Directory State Transitions for Read Requests by a Local Caching Agent			
GPIO Request Type	MEM Request Type	Current Directory State	New Directory State
RdCur, RdCode, RdData, RdDataMig RdInvOwn	MemRd	I/S	—
		A	*
	MemRdXtoI	I	—
InvItoE, InvXtoI NonSnpRd		S/A	I
	Mem InvXtoI	I/S/A	I
	MemRd	I/S/A	—

TABLE 5

Directory State Transitions for Read Requests by a Remote Caching Agent			
GPIO Request Type	MEM Request Type	Current Directory State	New Directory State
RdCur	MemRd	I/S	—
		A	*
RdCode	MemRdXtoS	I	S
		S	—
RdData, RdDataMig	MemRdData	A	S
		I	A
RdInvOwn	MemRdXtoA	S	—
		A	*
InvItoE	Mem InvXtoA	I/S	A
		A	—
InvXtoI	Mem InvXtoI	I	—
		S/A	I
NonSnpRd	MemRd	I/S/A	—

As shown in FIGS. 16A-16C, in some implementations, a cache line 1630 can include a field or bits (e.g., 1635) encoded to identify the directory state of the cache line 1630. In response to a read request, a copy of the cache line 1630 can be returned to the requesting host (e.g., 1305), the copy including the encoded directory state bits 1635. Turning to the example of FIG. 16B, a read transaction, in some cases, can result in a change to the directory state of the corresponding cache line. Indeed, Tables 4 and 5 outline the state transitions that can occur based on the type of the read request and the directory state of the requested cache line at the time of the read request. Further, in some implementations, the directory state transitions can further depend on whether the read request originates from a local or remote

caching agent. For instance, an implementation can use directory states to primarily track remote agents (e.g., whether the remote agents have shared copies, exclusive copies, etc.). In some implementations, the home agent can only be aware of directory covered agents (e.g., Remote agents) or agents that are not covered by the directory (e.g., Local agents), causing these determinations to be made for one or other of remote or local agents only. The directory bits can reflect the directory state in the remote sockets and can be used to determine whether remote snoops should be performed. In some implementations, local agents can possess copies of the cache lines and read requests from local agents can typically cause a transition of the directory state to an invalid (“I”), or “clean,” state.

As an example, in the illustration of FIG. 16B, a read request 1610 can result in a copy 1630 being retrieved (at 1620) by the buffer chip 1315 memory controller and forwarded (at 1640) (over a buffered memory link) to a host 1305. In this example, the directory state of the cache line 1630 at the time the request 1610 is received can be “A”. The directory state can be encoded in directory state bits 1635. Upon receiving the cache line 1630, the host 1305 may change the directory state. In some cases, the change can be based on the result of snoops performed by the home agent of the host. For instance, when the request is a “RdCur” read request, the host 1305 may snoop remote caching agents in connection with the request 1610. In the example of FIG. 16B, based on the results of a snoop, the host determines that the directory state should be updated to “S” (as reflected in bits 1635a). To allow this directory update to be communicated to the buffered memory, the host can writeback 1650 the cache line 1630 with the updated directory state bits 1635a. The memory controller of the buffer chip can then write 1655 the updated cache line with the updated directory state bits 1635 to memory.

In some cases, the memory controller of the buffer chip 1315 can infer the change to the directory state without the change being communicated back to the buffer chip (e.g., through a writeback (e.g., 1650)). For instance, as shown in Tables 4 and 5, some combinations of read request type and current directory state will always result in the same directory change (or lack of change). In Tables 4 and 5, the column “New Directory State” shows the directory state resulting from a read request of a particular type when the cache line has a particular directory state (and dependent on whether the request originates from a local or remote agent). As an example, as shown in Table 5, when the read request is from a remote agent and is a “RdData” request while the directory state is in an “I” state, the change to the directory state to an “A” state can be reliably predicted, among other examples, including those shown in Tables 4 and 5.

In Tables 4 and 5, when the “New Directory State” is indicated as “—” it can be predicted that no change to the directory state will result based on the read request. When the “New Directory State” is indicated as “*”, a change is possible but not predictable. For instance, any directory state changes may be dependent on results of a snoop performed by the requesting host. As an example, in Table 5, when the read request is from a remote agent and is a “RdData” request while the directory state is in an “A” state, a snoop may be performed by the home agent resulting in a potential unpredictable change to the directory state of the cache line. Accordingly, in cases indicated by “*”, the directory state may sometimes only be updated by the memory controller through a writeback or other explicit communication from the home agent. In cases where the new directory state is

31

predictable, the memory controller can update the directory state directly, without intervention of the home agent.

To illustrate, in FIG. 16C, an example is shown where a read request 1610 is received when the cache line is in an “A” state (as recorded in directory state bits 1635). In this example, based on the type of the read request 1610 and the cache line being in the “A” state (and based on the source of the request), the memory controller can determine that home agent will update the directory state to “I” as shown. The memory controller can identify the type of the read request and the source of the request based on fields of the read request and conclude, from this information, what the directory state will change to. The memory controller can preemptively change the directory state bits of the cache line copy in memory by writing (at 1645) a new copy of the cache line with the new directory state bits (1635b) to memory (at 1665) to reflect the change that the home agent will make to the directory state (at 1660). The memory controller of the buffer chip 1315 can thus be equipped with logic to determine whether a directory state change can be predicted from a particular read request type from a particular caching agent while the cache line is in a particular directory state. The memory controller can likewise determine whether it should wait and rely on a communication from the home agent to identify an unpredictable change to the directory state. Likewise, the home agent can include logic to determine when it should writeback (or otherwise communicate) updated directory state bits to facilitate updating of the cache line’s directory state as recorded in memory, or whether the home agent can forego any such communication based on an understanding that the separate memory controller will perform these updates directly. Both the memory controller and home agent can include or access data structures with machine readable data embodying information describing the combinations of directory state changes, such as described in the example Tables 4 and 5, among other examples.

FIG. 17 is a simplified representation 1700 of a cache line 1630, in accordance with some examples. In this particular example, the cache line can include payload data 1705 as well as error correction code (ECC) data (e.g., in an ECC field 1710). In this example, the directory state bits 1635 can be included in, or rolled out in the ECC bits 1710, as stored in the DRAM. For instance, in one example, the directory bits can be two directory bits stored within a 64-byte cache line. The directory state bits 1635 can be set by the host device on writes, directly updated by the memory controller in some read transactions (e.g., as in the example of FIG. 16C), or be based on a communication by the host in other read transactions (e.g., as in the example of FIG. 16B). In one example, a two bit directory state bit implementation can be implemented in accordance with the example of Table 6 below:

TABLE 6

Directory State Encoding					
Dir [1]	Dir [0]	State	Description	Remote socket caches	Local socket caches
0	0	I	Invalid State. The cache line is not cached in the remote socket caches. May be in I, S, or F in the local socket caches.	Not cached	Any state

32

TABLE 6-continued

Directory State Encoding					
Dir [1]	Dir [0]	State	Description	Remote socket caches	Local socket caches
0	1	—	Not used.		
1	1	S	Shared. The cache line is either cached in the S or F state or not cached at all.	I, S, or F	I, S, or F
1	0	A	Any. The cache line could be in any cache state (M, E, F, S, or I) in a remote socket. It also could be in any state in the local socket.	Any State (M, E, F, S, or I)	Any State (M, E, F, S, or I)

Other implementations can support additional or alternative directory states. Further, other implementations may use additional directory state bits. Additionally, while Tables 4 and 5 outline various read request types and predicted directory state changes resulting from these read requests, other implementations may support alternative sets of read request types and predicted effects on directory state based on the read request types, the directory states used, and the source of the read request, among other alternatives and examples.

HPI and an associated buffered memory access protocol can be incorporated in any variety of computing devices and systems, including mainframes, server systems, personal computers, mobile computers (such as tablets, smartphones, personal digital systems, etc.), smart appliances, gaming or entertainment consoles and set top boxes, among other examples. For instance, referring to FIG. 18, shown is a block diagram of a second system 1800 in accordance with an embodiment of the present invention. As shown in FIG. 18, multiprocessor system 1800 is a point-to-point interconnect system, and includes a first processor 1870 and a second processor 1880 coupled via a point-to-point interconnect 1850. Each of processors 1870 and 1880 may be some version of a processor. In one embodiment, 1852 and 1854 are part of a serial, point-to-point coherent interconnect fabric, such as a high-performance architecture. As a result, the invention may be implemented within the QPI architecture.

While shown with only two processors 1870, 1880, it is to be understood that the scope of the present invention is not so limited. In other embodiments, one or more additional processors may be present in a given processor. Processors 1870 and 1880 are shown including integrated memory controller units 1872 and 1882, respectively. Processor 1870 also includes as part of its bus controller units point-to-point (P-P) interfaces 1876 and 1878; similarly, second processor 1880 includes P-P interfaces 1886 and 1888. Processors 1870, 1880 may exchange information via a point-to-point (P-P) interface 1850 using P-P interface circuits 1878, 1888. As shown in FIG. 18, IMCs 1872 and 1882 couple the processors to respective memories, namely a memory 1832 and a memory 1834, which may be portions of main memory locally attached to the respective processors.

Processors 1870, 1880 each exchange information with a chipset 1890 via individual P-P interfaces 1852, 1854 using point to point interface circuits 1876, 1894, 1886, 1898. Chipset 1890 also exchanges information with a high-performance graphics circuit 1838 via an interface circuit 1892 along a high-performance graphics interconnect 1839.

A shared cache (not shown) may be included in either processor or outside of both processors; yet connected with the processors via P-P interconnect, such that either or both

processors' local cache information may be stored in the shared cache if a processor is placed into a low power mode.

Chipset **1890** may be coupled to a first bus **1816** via an interface **1896**. In one embodiment, first bus **1816** may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the present invention is not so limited.

As shown in FIG. **18**, various I/O devices **1814** are coupled to first bus **1816**, along with a bus bridge **1818** which couples first bus **1816** to a second bus **1820**. In one embodiment, second bus **1820** includes a low pin count (LPC) bus. Various devices are coupled to second bus **1820** including, for example, a keyboard and/or mouse **1822**, communication devices **1827** and a storage unit **1828** such as a disk drive or other mass storage device which often includes instructions/code and data **1830**, in one embodiment. Further, an audio I/O **1824** is shown coupled to second bus **1820**. Note that other architectures are possible, where the included components and interconnect architectures vary. For example, instead of the point-to-point architecture of FIG. **18**, a system may implement a multi-drop bus or other such architecture.

Referring now to FIG. **19**, shown is a block diagram of an embodiment of a multicore processor. As shown in the embodiment of FIG. **19**, processor **1900** includes multiple domains. Specifically, a core domain **1930** includes a plurality of cores **1930A-1930N**, a graphics domain **1960** includes one or more graphics engines having a media engine **1965**, and a system agent domain **1910**.

In various embodiments, system agent domain **1910** handles power control events and power management, such that individual units of domains **1930** and **1960** (e.g. cores and/or graphics engines) are independently controllable to dynamically operate at an appropriate power mode/level (e.g. active, turbo, sleep, hibernate, deep sleep, or other Advanced Configuration Power Interface like state) in light of the activity (or inactivity) occurring in the given unit. Each of domains **1930** and **1960** may operate at different voltage and/or power, and furthermore the individual units within the domains each potentially operate at an independent frequency and voltage. Note that while only shown with three domains, understand the scope of the present invention is not limited in this regard and additional domains may be present in other embodiments.

As shown, each core **1930** further includes low level caches in addition to various execution units and additional processing elements. Here, the various cores are coupled to each other and to a shared cache memory that is formed of a plurality of units or slices of a last level cache (LLC) **1940A-1940N**; these LLCs often include storage and cache controller functionality and are shared amongst the cores, as well as potentially among the graphics engine too.

As seen, a ring interconnect **1950** couples the cores together, and provides interconnection between the core domain **1930**, graphics domain **1960** and system agent circuitry **1910**, via a plurality of ring stops **1952A-1952N**, each at a coupling between a core and LLC slice. As seen in FIG. **19**, interconnect **1950** is used to carry various information, including address information, data information, acknowledgement information, and snoop/invalid information. Although a ring interconnect is illustrated, any known on-die interconnect or fabric may be utilized. As an illustrative example, some of the fabrics discussed above (e.g. another on-die interconnect, On-chip System Fabric (OSF), an Advanced Microcontroller Bus Architecture (AMBA)

interconnect, a multi-dimensional mesh fabric, or other known interconnect architecture) may be utilized in a similar fashion.

As further depicted, system agent domain **1910** includes display engine **1912** which is to provide control of and an interface to an associated display. System agent domain **1910** may include other units, such as: an integrated memory controller **1920** that provides for an interface to a system memory (e.g., a DRAM implemented with multiple DIMMs; coherence logic **1922** to perform memory coherence operations. Multiple interfaces may be present to enable interconnection between the processor and other circuitry. For example, in one embodiment at least one direct media interface (DMI) **1916** interface is provided as well as one or more PCIe™ interfaces **1914**. The display engine and these interfaces typically couple to memory via a PCIe™ bridge **1918**. Still further, to provide for communications between other agents, such as additional processors or other circuitry, one or more other interfaces may be provided.

While the present invention has been described with respect to a limited number of embodiments, those skilled in the art will appreciate numerous modifications and variations therefrom. It is intended that the appended claims cover all such modifications and variations as fall within the true spirit and scope of this present invention.

A design may go through various stages, from creation to simulation to fabrication. Data representing a design may represent the design in a number of manners. First, as is useful in simulations, the hardware may be represented using a hardware description language (HDL) or another functional description language. Additionally, a circuit level model with logic and/or transistor gates may be produced at some stages of the design process. Furthermore, most designs, at some stage, reach a level of data representing the physical placement of various devices in the hardware model. In the case where conventional semiconductor fabrication techniques are used, the data representing the hardware model may be the data specifying the presence or absence of various features on different mask layers for masks used to produce the integrated circuit. In some implementations, such data may be stored in a database file format such as Graphic Data System II (GDS II), Open Artwork System Interchange Standard (OASIS), or similar format.

In some implementations, software based hardware models, and HDL and other functional description language objects can include register transfer language (RTL) files, among other examples. Such objects can be machine-parsable such that a design tool can accept the HDL object (or model), parse the HDL object for attributes of the described hardware, and determine a physical circuit and/or on-chip layout from the object. The output of the design tool can be used to manufacture the physical device. For instance, a design tool can determine configurations of various hardware and/or firmware elements from the HDL object, such as bus widths, registers (including sizes and types), memory blocks, physical link paths, fabric topologies, among other attributes that would be implemented in order to realize the system modeled in the HDL object. Design tools can include tools for determining the topology and fabric configurations of system on chip (SoC) and other hardware device. In some instances, the HDL object can be used as the basis for developing models and design files that can be used by manufacturing equipment to manufacture the described hardware. Indeed, an HDL object itself can be provided as an input to manufacturing system software to cause the described hardware.

In any representation of the design, the data may be stored in any form of a machine readable medium. A memory or a magnetic or optical storage such as a disc may be the machine readable medium to store information transmitted via optical or electrical wave modulated or otherwise generated to transmit such information. When an electrical carrier wave indicating or carrying the code or design is transmitted, to the extent that copying, buffering, or re-transmission of the electrical signal is performed, a new copy is made. Thus, a communication provider or a network provider may store on a tangible, machine-readable medium, at least temporarily, an article, such as information encoded into a carrier wave, embodying techniques of embodiments of the present disclosure.

A module as used herein refers to any combination of hardware, software, and/or firmware. As an example, a module includes hardware, such as a micro-controller, associated with a non-transitory medium to store code adapted to be executed by the micro-controller. Therefore, reference to a module, in one embodiment, refers to the hardware, which is specifically configured to recognize and/or execute the code to be held on a non-transitory medium. Furthermore, in another embodiment, use of a module refers to the non-transitory medium including the code, which is specifically adapted to be executed by the microcontroller to perform predetermined operations. And as can be inferred, in yet another embodiment, the term module (in this example) may refer to the combination of the microcontroller and the non-transitory medium. Often module boundaries that are illustrated as separate commonly vary and potentially overlap. For example, a first and a second module may share hardware, software, firmware, or a combination thereof, while potentially retaining some independent hardware, software, or firmware. In one embodiment, use of the term logic includes hardware, such as transistors, registers, or other hardware, such as programmable logic devices.

Use of the phrase ‘configured to,’ in one embodiment, refers to arranging, putting together, manufacturing, offering to sell, importing and/or designing an apparatus, hardware, logic, or element to perform a designated or determined task. In this example, an apparatus or element thereof that is not operating is still ‘configured to’ perform a designated task if it is designed, coupled, and/or interconnected to perform said designated task. As a purely illustrative example, a logic gate may provide a 0 or a 1 during operation. But a logic gate ‘configured to’ provide an enable signal to a clock does not include every potential logic gate that may provide a 1 or 0. Instead, the logic gate is one coupled in some manner that during operation the 1 or 0 output is to enable the clock. Note once again that use of the term ‘configured to’ does not require operation, but instead focus on the latent state of an apparatus, hardware, and/or element, where in the latent state the apparatus, hardware, and/or element is designed to perform a particular task when the apparatus, hardware, and/or element is operating.

Furthermore, use of the phrases ‘to,’ ‘capable of/to,’ and/or ‘operable to,’ in one embodiment, refers to some apparatus, logic, hardware, and/or element designed in such a way to enable use of the apparatus, logic, hardware, and/or element in a specified manner. Note as above that use of to, capable to, or operable to, in one embodiment, refers to the latent state of an apparatus, logic, hardware, and/or element, where the apparatus, logic, hardware, and/or element is not operating but is designed in such a manner to enable use of an apparatus in a specified manner.

A value, as used herein, includes any known representation of a number, a state, a logical state, or a binary logical

state. Often, the use of logic levels, logic values, or logical values is also referred to as 1’s and 0’s, which simply represents binary logic states. For example, a 1 refers to a high logic level and 0 refers to a low logic level. In one embodiment, a storage cell, such as a transistor or flash cell, may be capable of holding a single logical value or multiple logical values. However, other representations of values in computer systems have been used. For example the decimal number ten may also be represented as a binary value of 1010 and a hexadecimal letter A. Therefore, a value includes any representation of information capable of being held in a computer system.

Moreover, states may be represented by values or portions of values. As an example, a first value, such as a logical one, may represent a default or initial state, while a second value, such as a logical zero, may represent a non-default state. In addition, the terms reset and set, in one embodiment, refer to a default and an updated value or state, respectively. For example, a default value potentially includes a high logical value, i.e. reset, while an updated value potentially includes a low logical value, i.e. set. Note that any combination of values may be utilized to represent any number of states.

The embodiments of methods, hardware, software, firmware or code set forth above may be implemented via instructions or code stored on a machine-accessible, machine readable, computer accessible, or computer readable medium which are executable by a processing element. A non-transitory machine-accessible/readable medium includes any mechanism that provides (i.e., stores and/or transmits) information in a form readable by a machine, such as a computer or electronic system. For example, a non-transitory machine-accessible medium includes random-access memory (RAM), such as static RAM (SRAM) or dynamic RAM (DRAM); ROM; magnetic or optical storage medium; flash memory devices; electrical storage devices; optical storage devices; acoustical storage devices; other form of storage devices for holding information received from transitory (propagated) signals (e.g., carrier waves, infrared signals, digital signals); etc., which are to be distinguished from the non-transitory mediums that may receive information there from.

Instructions used to program logic to perform embodiments of the invention may be stored within a memory in the system, such as DRAM, cache, flash memory, or other storage. Furthermore, the instructions can be distributed via a network or by way of other computer readable media. Thus a machine-readable medium may include any mechanism for storing or transmitting information in a form readable by a machine (e.g., a computer), but is not limited to, floppy diskettes, optical disks, Compact Disc, Read-Only Memory (CD-ROMs), and magneto-optical disks, Read-Only Memory (ROMs), Random Access Memory (RAM), Erasable Programmable Read-Only Memory (EPROM), Electrically Erasable Programmable Read-Only Memory (EEPROM), magnetic or optical cards, flash memory, or a tangible, machine-readable storage used in the transmission of information over the Internet via electrical, optical, acoustical or other forms of propagated signals (e.g., carrier waves, infrared signals, digital signals, etc.). Accordingly, the computer-readable medium includes any type of tangible machine-readable medium suitable for storing or transmitting electronic instructions or information in a form readable by a machine (e.g., a computer).

The following examples pertain to embodiments in accordance with this Specification. One or more embodiments may provide an apparatus, a system, a machine readable storage, a machine readable medium, hardware- and/or

software-based logic, and a method to receive a request over a link that requests a particular line in memory, identify a directory state record in memory that identifies a directory state of the particular line, identify a type of the request, determine that the directory state of the particular line is to change from the particular state to a new state based on the directory state of particular line and the type of the request, and change the directory state record, in response to receipt of the request, to reflect the new state. A copy of the particular line can be sent in response to the request.

In one example, the method is performed at least in part by a memory controller.

In one example, the request is to be received from a host device separate from the memory controller.

In one example, the request is to be received from the host device over a buffered memory access link.

In one example, the memory controller is provided on a buffer chip separate from the host device.

In one example, the request originates from a particular caching agent, and the change is to be determined from a type of the particular caching agent.

In one example, the type of the particular caching agent is one of a local caching agent or a remote caching agent.

In one example, the request includes one of a read request and a memory invalidate request.

In one example, the directory state record includes two or more bits and the bits are to be included in the line and reflect the particular state.

In one example, the memory controller is to directly write the new state to the particular line in memory.

In one example, a home agent updates the directory state to the new state and the memory controller updates the directory state to the new state independent of the home agent.

In one example, the bits are to be included in error correction code bits of the particular line.

In one example, the memory controller is to determine that directory state changes associated with another request are unpredictable.

In one example, the memory controller is further to receive a communication from a home agent to indicate a directory state change to the line corresponding to the other request and perform a write to the line corresponding to the other request to update the directory state of the line corresponding to the other request based on the communication.

In one example, the communication includes a writeback of the line corresponding to the other request, wherein the writtenback line is to indicate the directory state of the line corresponding to the other request.

In one example, the directory state of the line corresponding to the other request is to include a directory state record of the line corresponding to the other request.

In one example, the directory state is one of a set of directory states, and the set of directory states include a shared state, any state, and invalid state.

One or more embodiments may provide an apparatus, a system, a machine readable storage, a machine readable medium, a method, and hardware- and/or software-based logic (e.g., implementing caching agent logic) to determine a buffered memory request corresponding to a particular line of memory and of a particular one of a plurality of request types, send the request to a buffer chip over a buffered memory link, receive a copy of the particular line of memory in response to the request, determine from a directory state record included in the particular line that the particular line is in a particular directory state, determine a change to the directory state, and determine whether to communicate the

change to the buffer chip based on the particular directory state and particular request type of the request.

In one example, the change is not to be communicated to the buffer chip if the change is predictable from the particular directory state and the particular request type of the request.

In one example, the change is to be communicated to the buffer chip if the change is unpredictable by the buffer chip from the particular directory state and the particular request type of the request.

One or more embodiments may provide a system that includes a buffer chip, memory to be accessed by through the buffer chip, and a processor block coupled to the buffer chip by a memory access link. The buffer chip includes a memory controller to receive a request from the processor block over the memory access link related to a particular line in the memory, identify a directory state record in the memory that identifies a directory state of the particular line, identify a type of the request from the request, determine that the directory state of the particular line is to change from the particular state to a new state based on the directory state of the particular line and the type of the request, and change the directory state record, in response to receipt of the request, to reflect the new state.

In one example, the processor block is to interface with one or more other devices over a general purpose input/output (GPIO) interconnect link, the memory access link is different from the GPIO interconnect link, and a physical layer of the memory access link is based on a physical layer of the GPIO interconnect link.

Reference throughout this specification to “one embodiment” or “an embodiment” means that a particular feature, structure, or characteristic described in connection with the embodiment is included in at least one embodiment of the present invention. Thus, the appearances of the phrases “in one embodiment” or “in an embodiment” in various places throughout this specification are not necessarily all referring to the same embodiment. Furthermore, the particular features, structures, or characteristics may be combined in any suitable manner in one or more embodiments.

In the foregoing specification, a detailed description has been given with reference to specific exemplary embodiments. It will, however, be evident that various modifications and changes may be made thereto without departing from the broader spirit and scope of the invention as set forth in the appended claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense. Furthermore, the foregoing use of embodiment and other exemplarily language does not necessarily refer to the same embodiment or the same example, but may refer to different and distinct embodiments, as well as potentially the same embodiment.

What is claimed is:

1. An apparatus comprising:

a controller, coupled to a memory, wherein the controller comprises an interface to couple to a processor device and the interface comprises circuitry to:

receive a request flit from the processor device over an interconnect, the request flit is according to a flit format, wherein the request flit comprises a request associated with a line in the memory, the request flit comprises a field to identify a directory state to be applied to the line in association with the request, wherein the flit format defines a flit width and a plurality of slots to be included in flits based on the flit format; and

39

send a response flit to the request on the interconnect, wherein the response flit is according to the flit format.

2. The apparatus of claim 1, wherein the response flit comprises response data from the line.

3. The apparatus of claim 2, wherein at least one of the plurality of slots of the response flit comprises an all-data slot.

4. The apparatus of claim 1, wherein the response flit comprises a no data response.

5. The apparatus of claim 1, wherein the request comprises an invalidation request.

6. The apparatus of claim 1, wherein the directory state identified in the field comprises one of an Any (A), Invalid (I), or Shared(S) directory state.

7. The apparatus of claim 1, wherein the flit format defines a cyclic redundancy check (CRC) field to carry a CRC value for the flits based on the flit format.

8. The apparatus of claim 1, wherein the request flit comprises a source identifier field to carry an identifier associated with the processor device.

9. The apparatus of claim 8, wherein the response flit comprises a destination identifier field to carry the identifier associated with the processor device.

10. The apparatus of claim 1, wherein the flit width is at least 68 bits.

11. The apparatus of claim 1, wherein the flit width is at least 192 bits.

12. The apparatus of claim 1, wherein the plurality of slots comprises at least three slots.

13. The apparatus of claim 1, wherein the interconnect comprises a physical layer and the physical layer is based on a protocol to support link layer data of multiple different protocols, wherein the multiple different protocols comprise a memory protocol and at least one other different interconnect protocol, and the request flit and the response flit are based on the memory protocol.

14. The apparatus of claim 13, wherein the flit format is also used in flits defined in the other protocol.

15. The apparatus of claim 13, wherein the other different interconnect protocol comprises an input/output (I/O) interconnect protocol.

16. An apparatus comprising:

a processor;

a port to couple the processor to a memory device over an interconnect, wherein the interconnect comprises a physical layer and the physical layer is based on a

40

protocol to support link layer data of multiple different protocols, wherein the multiple different protocols comprise a memory protocol and at least one other different interconnect protocol, wherein the port comprises circuitry to support a link layer of at least the memory protocol, and the circuitry is to:

identify a memory request associated with memory attached to the memory device;

generate a flit based on the memory protocol, wherein the flit comprises a plurality of slots, a CRC field to carry a CRC value for the flit, request information associated with the memory request, a source identifier field to carry an identifier associated with the processor, and a directory state field to identify a directory state to be applied to a line in the memory in association with the memory request; and

send the flit to the memory device on the interconnect.

17. The apparatus of claim 16, further comprising protocol circuitry to generate link layer data according to the other different interconnect protocol.

18. A system comprising:

a processor device; and

a memory device coupled to the processor device by an interconnect, wherein the memory device is coupled to a memory and comprises:

an interface to couple the memory device to the interconnect, the interface comprising circuitry to:

receive a request flit from the processor device over the interconnect, the request flit is according to a flit format, wherein the request flit comprises a request associated with a line in the memory, the request flit comprises a field to identify a directory state to be applied to the line in association with the request, wherein the flit format defines a flit width and a plurality of slots to be included in flits based on the flit format; and

send a response flit to the processor device on the interconnect, wherein the response flit is according to the flit format.

19. The system of claim 18, wherein the memory device comprises the memory.

20. The system of claim 18, wherein the memory is external to the memory device.

* * * * *